เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

บทที่ 2 เรื่องข้อมูลและระบบตัวเลข

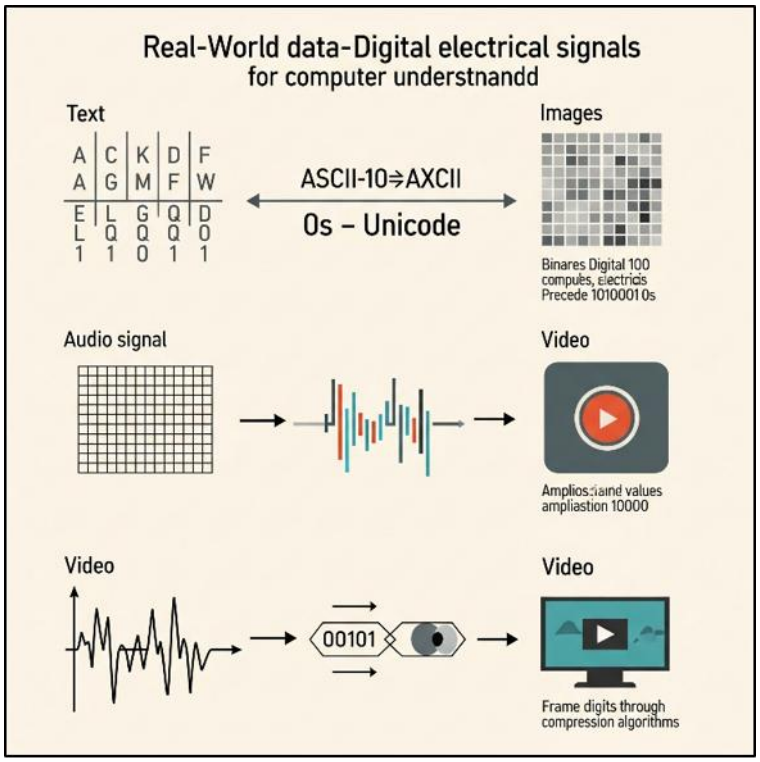
วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. อธิบายความหมายข้อมูลคอมพิวเตอร์ใช้ประมวลผลและเชื่อมโยงกับสัญญาณดิจิทัล (Logic 0 และ 1)
2. อธิบายกระบวนการเข้ารหัส (Encoding) และถอดรหัส (Decoding) ได้อย่างถูกต้อง
3. อธิบายวัฏจักรการไหลของข้อมูลในระบบคอมพิวเตอร์ได้
4. อธิบายระบบตัวเลข เลขฐาน 2 (Binary) และเลขฐาน 8 (Octal) ได้
5. อธิบายระบบตัวเลข เลขฐาน 10 (Decimal), และเลขฐาน 16 (Hexadecimal) ได้
6. อธิบายความสัมพันธ์และลักษณะเฉพาะของระบบตัวเลขในคอมพิวเตอร์ได้
7. อธิบายการวัดขนาดข้อมูล (Data Size Measurement) และความสัมพันธ์ระหว่าง บิต และ ไบต์ ได้
8. อธิบายหลักการแปลงเลขฐาน 2 (Binary) ไปเป็น เลขฐาน 10 (Decimal) ได้อย่างถูกต้อง
9. อธิบายหลักการแปลงเลขฐาน 8 (Octal) ไปเป็น เลขฐาน 10 (Decimal) ได้อย่างถูกต้อง
10. อธิบายหลักการแปลงเลขฐาน 16 (Hexadecimal) ไปเป็น เลขฐาน 10 (Decimal) ได้อย่างถูกต้อง
11. อธิบายหลักการแปลงเลขฐาน 10 (Decimal) ไปเป็น เลขฐาน 2 (Binary) ได้อย่างถูกต้อง
12. อธิบายหลักการแปลงเลขฐาน 10 (Decimal) ไปเป็น เลขฐาน 8 (Octal) ได้อย่างถูกต้อง
13. อธิบายหลักการแปลงเลขฐาน 10 (Decimal) ไปเป็น เลขฐาน 16 (Hexadecimal) ได้อย่างถูกต้อง
14. อธิบายหลักการแปลงเลขฐาน 2 (Binary) ไปเป็นเลขฐาน 8 (Octal) แบบโดยตรงได้อย่างถูกต้อง
15. อธิบายหลักการแปลงเลขฐาน 8 (Octal) ไปเป็นเลขฐาน 2 (Binary) แบบโดยตรงได้อย่างถูกต้อง
16. อธิบายหลักการแปลงเลขฐาน 2 (Binary) ไปเป็นเลขฐาน 16 (Hexadecimal) แบบโดยตรงได้อย่างถูกต้อง
17. อธิบายหลักการแปลงเลขฐาน 16 (Hexadecimal) ไปเป็นเลขฐาน 2 (Binary) แบบโดยตรงได้อย่างถูกต้อง
18. อธิบายการคำนวณทางคณิตศาสตร์ การบวกเลขฐาน 2 (Binary Addition) ได้อย่างถูกต้อง
19. อธิบายการคำนวณทางคณิตศาสตร์ การลบเลขฐาน 2 (Binary Subtraction) ได้อย่างถูกต้อง
20. อธิบายการคำนวณทางคณิตศาสตร์ การบวกเลขฐาน 8 (Octal Addition) ได้อย่างถูกต้อง
21. อธิบายการคำนวณทางคณิตศาสตร์ การลบเลขฐาน 8 (Octal Addition) ได้อย่างถูกต้อง
22. อธิบายการคำนวณทางคณิตศาสตร์ การบวกเลขฐานสิบหก (Hexadecimal Addition) ได้อย่างถูกต้อง
23. อธิบายการคำนวณทางคณิตศาสตร์ การบวกเลขฐานสิบหก (Hexadecimal Addition) ได้อย่างถูกต้อง
24. อธิบายวิธีการประยุกต์ใช้การลบเลขฐานสองด้วย 1's Complement ในการคำนวณได้อย่างถูกต้อง
25. อธิบายวิธีการประยุกต์ใช้การลบเลขฐานสองด้วย 2's Complement ในการคำนวณได้อย่างถูกต้อง

26. อธิบายหลักการแทนค่าข้อมูลด้วยรหัส (Data Encoding) ได้
27. อธิบายตารางการเปรียบเทียบรหัส ASCII ได้
28. อธิบายหลักการใช้งานของรหัส ASCII ในการแทนข้อมูลประเภทตัวเลข ตัวอักษร และสัญลักษณ์ได้

1.ความหมายข้อมูลคอมพิวเตอร์ใช้ประมวลผลและเชื่อมโยงกับสัญญาณดิจิทัล (Logic 0 และ 1)

ในการทำความเข้าใจการทำงานของคอมพิวเตอร์ สิ่งที่สำคัญที่สุดคือการเข้าใจว่าข้อมูลที่เราใช้งานในชีวิตประจำวัน ไม่ว่าจะเป็นตัวอักษรที่เราพิมพ์ รูปภาพที่เราเห็น เสียงเพลงที่เราได้ยิน หรือภาพยนตร์ที่เราชม ล้วนต้องถูกแปลงให้อยู่ในรูปแบบที่เครื่องจักรเข้าใจได้ ซึ่งก็คือ **สัญญาณไฟฟ้าดิจิทัล** อันประกอบด้วยเพียงสองสถานะคือ "0" และ "1"



รูปที่ 2-1 กระบวนการเข้ารหัสและถอดรหัสข้อมูลในระบบคอมพิวเตอร์

1. ข้อมูลในรูปแบบดิจิทัล "0" และ "1": ภาษาของคอมพิวเตอร์

คอมพิวเตอร์เป็นอุปกรณ์อิเล็กทรอนิกส์ที่ทำงานด้วยการควบคุมกระแสไฟฟ้า ดังนั้นสถานะพื้นฐานที่คอมพิวเตอร์รับรู้ได้จึงมีเพียงสองสถานะเท่านั้น ซึ่งเราใช้ "0" และ "1" ในการแทนค่าสถานะเหล่านี้:

0 (ศูนย์): แทนสถานะที่ **ไม่มีกระแสไฟฟ้า** ไหลผ่าน (หรือแรงดันไฟฟ้าต่ำ)

1 (หนึ่ง): แทนสถานะที่ **มีกระแสไฟฟ้า** ไหลผ่าน (หรือแรงดันไฟฟ้าสูง)

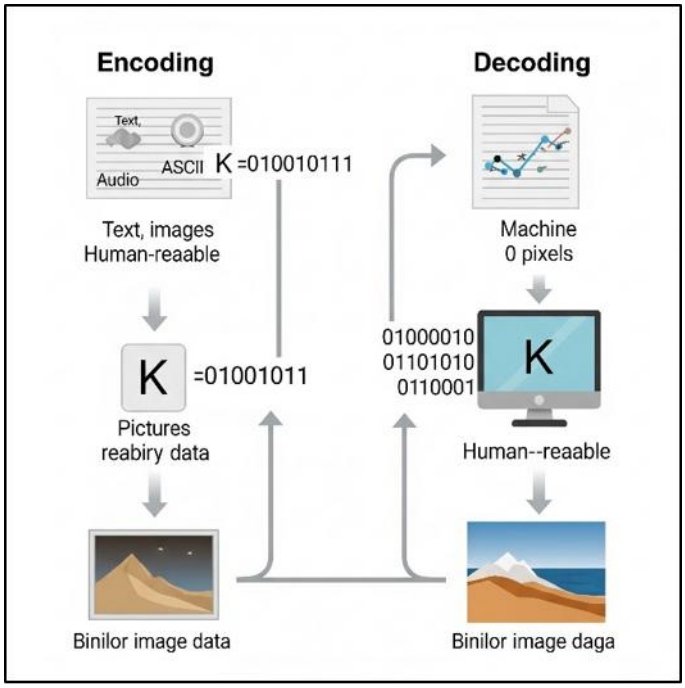
หน่วยที่เล็กที่สุดของข้อมูลในระบบคอมพิวเตอร์คือ **บิต (Bit)** ซึ่งย่อมาจาก **Binary Digit** บิตหนึ่งบิตสามารถเก็บค่าได้เพียง 0 หรือ 1 เท่านั้นเพื่อที่จะแทนข้อมูลที่มีความหมายและซับซ้อนมากขึ้น บิตหลายๆ บิตจะถูกนำมารวมกันเป็นกลุ่ม เช่น:

- **1 ไบต์ (Byte) = 8 บิต**: สามารถใช้แทนตัวอักษรหนึ่งตัว (เช่น 'A', 'บ'), ตัวเลข, หรือสัญลักษณ์พิเศษหนึ่งตัวได้ เนื่องจาก 8 บิตสามารถสร้างรูปแบบที่แตกต่างกันได้ถึง 28=256 รูปแบบ ซึ่งเพียงพอสำหรับการแทนตัวอักษร ตัวเลข และสัญลักษณ์ส่วนใหญ่
- ข้อมูลที่มีขนาดใหญ่ขึ้น เช่น รูปภาพ, เสียง, วิดีโอ ก็จะประกอบไปด้วยไบต์จำนวนมหาศาลที่เรียงต่อกันเป็นชุด เพื่อให้คอมพิวเตอร์สามารถเก็บและประมวลผลได้

ดังนั้น ไม่ว่าข้อมูลนั้นจะมาจากไหน หรือเป็นข้อมูลประเภทใด สุดท้ายแล้วเมื่อเข้าสู่ระบบคอมพิวเตอร์ มันจะถูกแปลงเป็นชุดของ 0 และ 1 เสมอ

2. กระบวนการเข้ารหัส (Encoding) และถอดรหัส (Decoding) ข้อมูล

สะพานเชื่อมระหว่างมนุษย์และเครื่องจักรเนื่องจากมนุษย์เราสื่อสารกันด้วยภาษาธรรมชาติ ตัวเลขที่ใช้กันทั่วไป (เลขฐานสิบ) และสื่อต่างๆ ที่เราคุ้นเคย ในขณะที่คอมพิวเตอร์เข้าใจเพียง 0 และ 1 เท่านั้น จึงจำเป็นต้องมี "สะพาน" ที่ทำหน้าที่แปลงข้อมูลไปมา กระบวนการนี้คือ การเข้ารหัสและการถอดรหัส



รูปที่ 2-2 กระบวนการเข้ารหัส (Encoding) และถอดรหัส (Decoding) ในระบบคอมพิวเตอร์

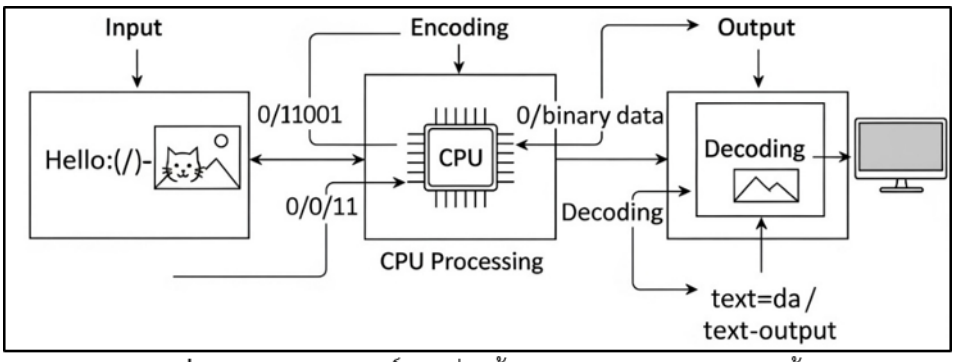
การเข้ารหัส (Encoding):

- **หน้าที่**: เป็นกระบวนการแปลงข้อมูลจาก "ภาษาคน" (Human-Readable Format) ให้เป็น "ภาษาคอมพิวเตอร์" (Machine-Readable Binary Code) หรือก็คือชุดของ 0 และ 1
- **เหตุผล**: คอมพิวเตอร์ไม่สามารถประมวลผลตัวอักษร 'K' หรือรูปภาพใบหน้าได้โดยตรง แต่จะเข้าใจเพียงสถานะทางไฟฟ้า 0 และ 1 เท่านั้น การเข้ารหัสจึงทำให้ข้อมูลเหล่านั้นสามารถถูกจัดเก็บ, ส่งผ่าน, และประมวลผลในหน่วยต่างๆ ของคอมพิวเตอร์ได้
- **ตัวอย่าง**:
  - เมื่อเราพิมพ์ตัวอักษร 'K' บนแป้นพิมพ์ ตัวอักษร 'K' จะถูกเข้ารหัสตามมาตรฐานที่กำหนด (เช่น รหัส ASCII หรือ Unicode) ให้กลายเป็นชุดของเลขฐานสองที่แน่นอน เช่น 01001011 (สำหรับ ASCII) จากนั้นชุดเลขฐานสองนี้จะถูกส่งไปยังหน่วยความจำหรือ CPU เพื่อประมวลผล
  - รูปภาพจะถูกแปลงเป็นชุดของพิกเซล (Pixel) แต่ละพิกเซลจะมีค่าสีที่ถูกเข้ารหัสเป็นชุดของ 0 และ 1

- **การถอดรหัส (Decoding)**:
  - **หน้าที่**: เป็นกระบวนการย้อนกลับ คือการแปลงข้อมูลจาก "ภาษาคอมพิวเตอร์" (ชุดของ 0 และ 1 ที่ถูกประมวลผลแล้ว) ให้กลับมาเป็น "ภาษาคน" (Human-Readable Format) เพื่อให้เราสามารถเข้าใจและนำไปใช้งานได้
  - **เหตุผล**: เมื่อคอมพิวเตอร์ประมวลผลข้อมูลเสร็จสิ้น ผลลัพธ์ที่ได้ยังคงอยู่ในรูปของ 0 และ 1 หากไม่มีการถอดรหัส เราก็จะไม่สามารถอ่านหรือมองเห็นผลลัพธ์เหล่านั้นได้

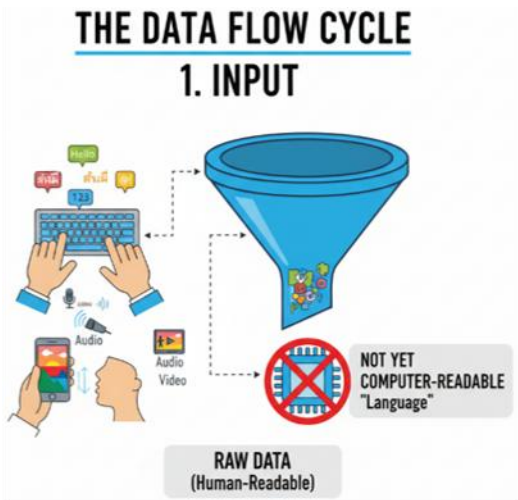
- **ตัวอย่าง**:
  - เมื่อ CPU ประมวลผลชุดเลขฐานสอง 01001011 เสร็จสิ้นและต้องการแสดงผลบนหน้าจอ ชิปควบคุมการแสดงผลจะทำการถอดรหัสชุด 01001011 นี้ให้กลับมาเป็นตัวอักษร 'K' แล้วแสดงผลบนหน้าจอให้เราเห็น
  - ชุดของ 0 และ 1 ที่แทนข้อมูลรูปภาพจะถูกถอดรหัสและแปลงเป็นสัญญาณไฟฟ้าที่ขับเคลื่อนพิกเซลบนหน้าจอ ทำให้เรามองเห็นเป็นภาพได้

สรุป เพราะฉะนั้นกระบวนการเข้ารหัสและถอดรหัสนี้เกิดขึ้นอย่างรวดเร็วและต่อเนื่องตลอดเวลาในระบบคอมพิวเตอร์ ทำให้เราสามารถโต้ตอบกับเครื่องจักรได้อย่างราบรื่นโดยไม่จำเป็นต้องรับรู้ถึงการทำงานของเลข 0 และ 1 ภายใน



รูปที่ 2-3 ความสัมพันธ์ระหว่างข้อมูล (Data), สัญญาณไฟฟ้า และผลลัพธ์ (Output)

วัฏจักรการไหลของข้อมูลในระบบคอมพิวเตอร์ : เป็นลำดับขั้นตอน 5 ขั้น ของการทำงานของคอมพิวเตอร์

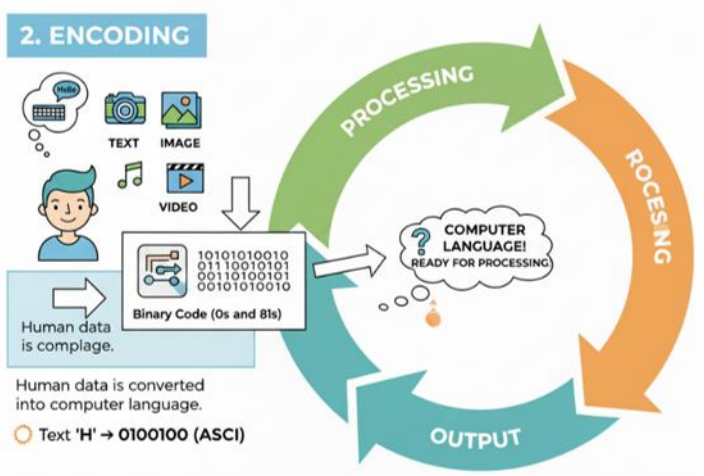


รูปที่ 2-4 การรับข้อมูลเข้าสู่คอมพิวเตอร์

1. **Input (ข้อมูลนำเข้า)**:

- นี่คือจุดเริ่มต้นที่ข้อมูลจากโลกภายนอก (หรือจากผู้ใช้งาน) เข้าสู่ระบบคอมพิวเตอร์
- ข้อมูลอยู่ในรูปแบบที่มนุษย์เข้าใจและใช้งานได้ง่าย เช่น:
  - **ข้อความ (Text)**: ตัวอักษร, ตัวเลข, สัญลักษณ์ต่างๆ ที่เราพิมพ์ผ่านแป้นพิมพ์ (เช่น "Hello", "สวัสดี")
  - **รูปภาพ (Image)**: ภาพถ่าย, ภาพวาด, กราฟิกต่างๆ ที่เราสแกนเข้ามา หรือสร้างขึ้นมา
  - **เสียง (Audio)**: เสียงพูด, เสียงเพลง ที่เราบันทึก
  - **วิดีโอ (Video)**: ภาพเคลื่อนไหวพร้อมเสียง

สรุป ณ ขั้นตอนนี้ คอมพิวเตอร์ยังไม่สามารถประมวลผลข้อมูลเหล่านี้ได้โดยตรง เพราะยังไม่ใช่ "ภาษา" ที่คอมพิวเตอร์เข้าใจ



รูปที่ 2-5 รูปกระบวนการเข้ารหัสข้อมูล

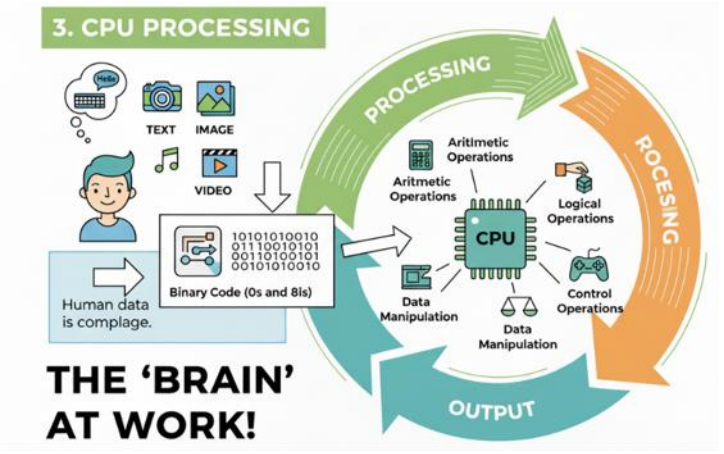
2. Encoding (การเข้ารหัส):

- นี่คือขั้นตอนสำคัญที่ทำหน้าที่เป็น "ล่าม" หรือ "ตัวแปลงภาษา"
- **กระบวนการ**: ข้อมูลที่อยู่ในรูปแบบที่มนุษย์เข้าใจจะถูกแปลงให้เป็นชุดของ **เลขฐานสอง (Binary Code)** ซึ่งประกอบด้วยเพียง "0" และ "1" เท่านั้น
- **เหตุผล**: คอมพิวเตอร์ทำงานด้วยวงจรอิเล็กทรอนิกส์ที่รับรู้เพียงสองสถานะ (เปิด/ปิด, มีไฟ/ไม่มีไฟ) ดังนั้นข้อมูลทุกชนิดจึงต้องถูกแปลงให้อยู่ในรูปแบบของสัญญาณไฟฟ้าดิจิทัลเหล่านี้ เพื่อให้หน่วยประมวลผลสามารถจัดการได้

ตัวอย่าง: ตัวอักษร 'H' อาจถูกเข้ารหัสเป็น 01001000 (ตามรหัส ASCII)

- สีของจุดภาพ (Pixel) ในรูปภาพจะถูกแปลงเป็นชุดของ 0s และ 1s ที่แทนค่าสีนั้นๆ

สรุป ข้อมูลที่ผ่านการเข้ารหัสแล้วจะกลายเป็น "ภาษาคอมพิวเตอร์" ที่พร้อมสำหรับการประมวลผล

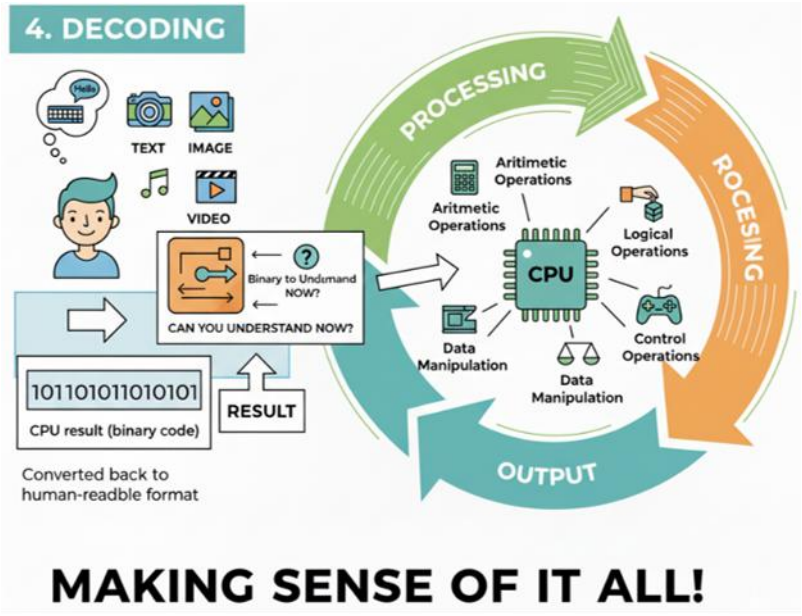


รูปที่ 2-6 กระบวนการประมวลผล

3. CPU Processing (การประมวลผลโดย CPU) :

- CPU (Central Processing Unit) หรือหน่วยประมวลผลกลาง คือ "สมอง" ของคอมพิวเตอร์
- **กระบวนการ**: ข้อมูลที่อยู่ในรูปของเลขฐานสองที่ผ่านการเข้ารหัสมาแล้ว จะถูกส่งมายัง CPU เพื่อดำเนินการตามคำสั่งต่างๆ
- **หน้าที่ของ CPU**:
  - **การคำนวณ (Arithmetic Operations)**: บวก, ลบ, คูณ, หาร
  - **การเปรียบเทียบ (Logical Operations)**: ตรวจสอบเงื่อนไขต่างๆ
  - **การจัดการข้อมูล (Data Manipulation)**: ย้ายข้อมูล, จัดเรียงข้อมูล
  - **การควบคุม (Control Operations)**: ควบคุมการทำงานของส่วนต่างๆ ในคอมพิวเตอร์

สรุป การประมวลผลทั้งหมดนี้เกิดขึ้นกับข้อมูลในรูปแบบของ “0” และ “1” เท่านั้น



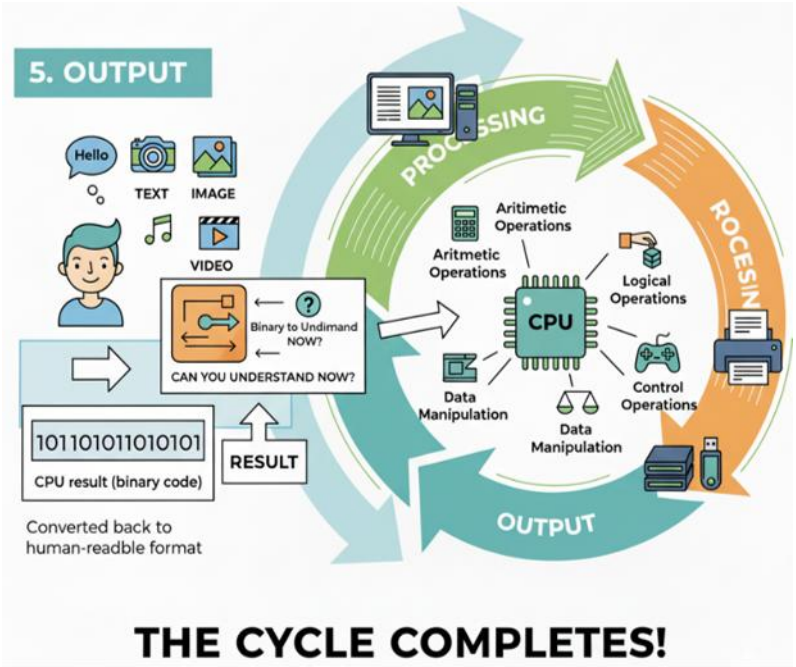
รูปที่ 2-7 การถอดรหัส (Decoding) กลับเป็นข้อมูลที่ใช้งานได้

4. Decoding (การถอดรหัส):

- นี่คือขั้นตอนย้อนกลับของการเข้ารหัส ทำหน้าที่เป็น "ล่าม" อีกครั้ง
- **กระบวนการ**: ผลลัพธ์ที่ได้จากการประมวลผลของ CPU ซึ่งยังคงอยู่ในรูปของเลขฐานสอง จะถูกแปลงกลับมาเป็นรูปแบบที่มนุษย์สามารถเข้าใจและรับรู้ได้
- **เหตุผล**: เพื่อให้ผู้ใช้งานสามารถเห็น, ได้ยิน, หรืออ่านผลลัพธ์ที่คอมพิวเตอร์ประมวลผลเสร็จสิ้น

สรุป

- *ชุดของ “0” และ “1” ที่แทนตัวอักษรจะถูกถอดรหัสกลับมาเป็นตัวอักษรที่แสดงบนหน้าจอ*
- *ชุดของ “0” และ “1” ที่แทนข้อมูลรูปภาพจะถูกถอดรหัสและส่งสัญญาณไปยังหน้าจอเพื่อแสดงผลเป็นภาพที่เรามองเห็น*



รูปที่ 2-8 การแสดงผลลัพธ์ (Output) จากการประมวลผลข้อมูล

5. Output (ข้อมูลส่งออก):

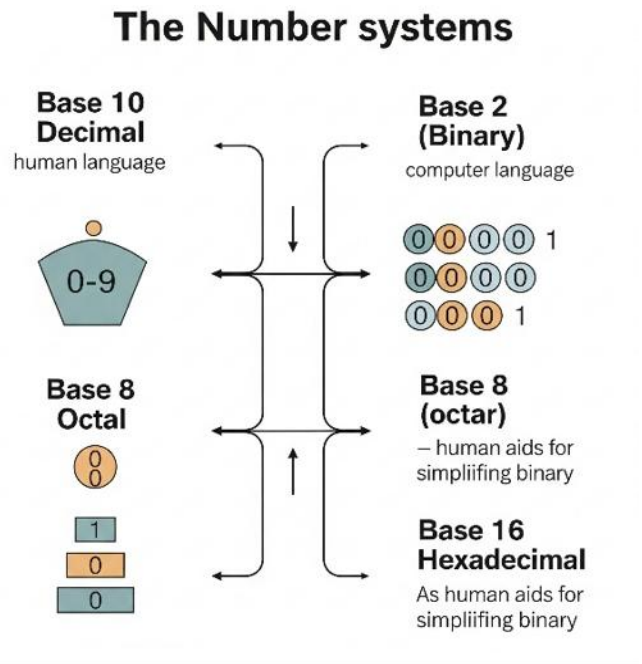
- นี่คือขั้นตอนสุดท้ายที่ข้อมูลถูกนำเสนอออกมาในรูปแบบที่มนุษย์เข้าใจ

สรุป

- *จอภาพ (Monitor): แสดงผลข้อความ, รูปภาพ, วิดีโอ*
- *เครื่องพิมพ์ (Printer): พิมพ์เอกสาร, รูปภาพ*
- *ลำโพง (Speaker): ส่งเสียง*
- *อุปกรณ์จัดเก็บข้อมูล (Storage Devices): บันทึกข้อมูลในรูปแบบที่สามารถนำกลับมาใช้ใหม่ได้*

3.ระบบตัวเลข (Number Systems)

ในชีวิตประจำวัน เราคุ้นเคยกับระบบเลขฐาน 10 (Decimal) เป็นอย่างดี แต่คอมพิวเตอร์นั้นจะใช้ระบบเลขฐาน 2 (Binary) เป็นหลักในการทำงาน ส่วนเลขฐาน 8 (Octal) และเลขฐาน 16 (Hexadecimal) จะเป็นระบบที่เข้ามาช่วยให้การทำงานกับเลขฐาน 2 เป็นเรื่องที่ง่ายขึ้นสำหรับมนุษย์



รูปที่ 2-9 ระบบตัวเลขที่ใช้ในระบบคอมพิวเตอร์

1. ระบบเลขฐาน 10 (Decimal Number System)

- **ลักษณะเด่น:** เป็นระบบตัวเลขที่เราใช้กันในชีวิตประจำวัน มีตัวเลขโดด 10 ตัว คือ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **ฐาน (Base/Radix):** 10
- **การแสดงค่า:** แต่ละหลักมีค่าประจำหลัก (Place Value) เป็นกำลังของ 10 เช่น
  - $123_{10}=(1\times10^2)+(2\times10^1)+(3\times10^{0)}$
  - หรือ $123_{10}=(1\times100)+(2\times10)+(3\times1)$
- **การใช้งาน:** ใช้ในการนับ, การคำนวณในชีวิตประจำวันของเรา

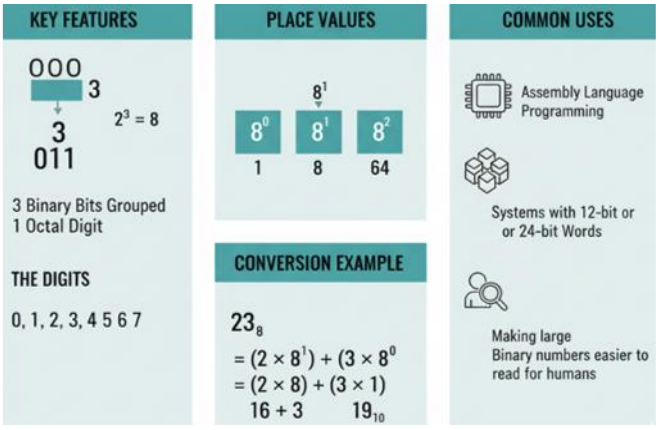
2. ระบบเลขฐาน 2 (Binary Number System)

- **ลักษณะเด่น:** เป็นระบบตัวเลขที่คอมพิวเตอร์ใช้ในการจัดเก็บและประมวลผลข้อมูล เพราะสามารถแทนสถานะทางไฟฟ้า 2 สถานะ (เปิด/ปิด, มีไฟ/ไม่มีไฟ) ได้อย่างชัดเจน
- **ฐาน (Base/Radix):** 2
- **ตัวเลขโดด:** มีเพียง 2 ตัว คือ **0** และ **1** (แต่ละหลักเรียกว่า "บิต - Bit")
- **การแสดงค่า:** แต่ละหลักมีค่าประจำหลักเป็นกำลังของ 2 เช่น
  - $1011_2=(1\times2^3)+(0\times2^2)+(1\times2^1)+(1\times2^0)$
  - $1011_2=(1\times8)+(0\times4)+(1\times2)+(1\times1)=8+0+2+1=11_{10}$
- **การใช้งาน:** เป็นภาษาพื้นฐานของคอมพิวเตอร์

3. ระบบเลขฐาน 8 (Octal Number System)

- **ลักษณะเด่น:** เป็นระบบที่สะดวกในการแปลงไปมาระหว่างเลขฐาน 2 และฐาน 8 ได้ง่าย เพราะ 23=8 หมายความว่า เลขฐาน 2 จำนวน 3 บิต สามารถแทนด้วยเลขฐาน 8 เพียง 1 หลัก

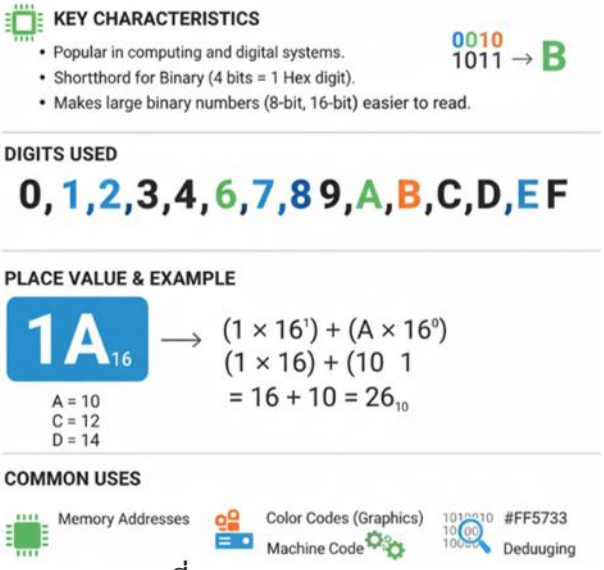
- **ฐาน (Base/Radix):** 8
- **ตัวเลขโดด:** 0, 1, 2, 3, 4, 5, 6, 7
- **การแสดงค่า:** แต่ละหลักมีค่าประจำหลักเป็นกำลังของ 8 เช่น
  - $23_8=(2\times8^1)+(3\times8^0)$
  - $23_8=(2\times8)+(3\times1)=16+3=19_{10}$
- **การใช้งาน:** เคยนิยมใช้ในการเขียนโปรแกรมภาษาแอสเซมบลี หรือแสดงค่าในระบบคอมพิวเตอร์บางประเภทที่คำมีความยาวบิตเป็นกลุ่มของ 3 บิต (เช่น ระบบที่ใช้ 12 บิต หรือ 24 บิต) เพื่อให้มนุษย์อ่านค่าเลขฐาน 2 จำนวนมากๆ ได้ง่ายขึ้น



รูปที่ 2-10 ระบบเลขฐาน 8 (Octal Number System)

4. ระบบเลขฐาน 16 (Hexadecimal Number System)

- **ลักษณะเด่น:** ระบบได้รับความนิยมอย่างมากในการทำงานกับคอมพิวเตอร์และระบบดิจิทัล เพราะ 24=16 หมายความว่า เลขฐาน 2 จำนวน 4 บิต สามารถแทนด้วยเลขฐาน 16 เพียง 1 หลัก ทำให้การเขียนและการอ่านค่าเลขฐาน 2 จำนวนมากๆ (โดยเฉพาะ 8 บิต หรือ 16 บิต) ง่ายและกระชับขึ้น
- **ฐาน (Base/Radix):** 16
- **ตัวเลขโดด:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, และใช้ตัวอักษร A, B, C, D, E, F แทนค่า 10 ถึง 15 ตามลำดับ:
  - A = 10
  - B = 11
  - C = 12
  - D = 13
  - E = 14
  - F = 15
- **การแสดงค่า:** แต่ละหลักมีค่าประจำหลักเป็นกำลังของ 16 เช่น
  - $1A_{16}=(1\times16^1)+(A\times16^0)$
  - $1A_{16}=(1\times16)+(10\times1)=16+10=26_{10}$
- **การใช้งาน:** ใช้กันอย่างแพร่หลายในการแสดงค่าที่อยู่หน่วยความจำ (Memory Address), ค่าสีในงานกราฟิก, รหัสเครื่อง (Machine Code), และในการดีบัก (Debug) โปรแกรม

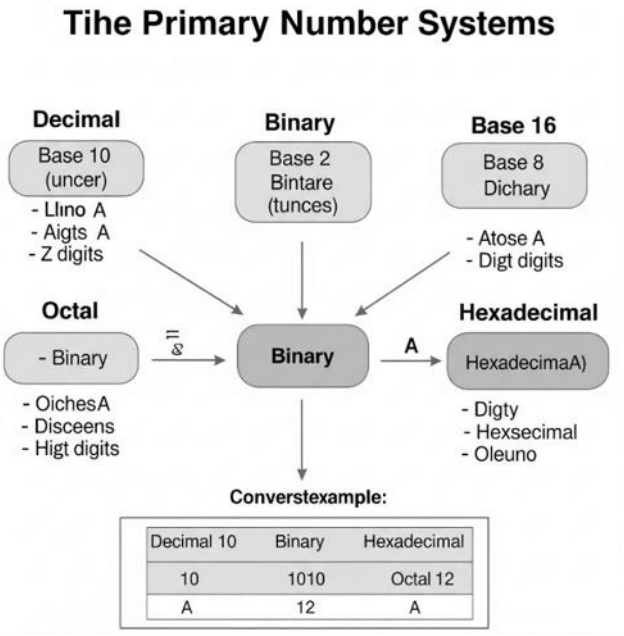


รูปที่ 2-11 ระบบเลขฐาน 16 (Hexadecimal Number System)

4.ความสัมพันธ์และลักษณะเฉพาะของระบบตัวเลขในคอมพิวเตอร์

1. ระบบเลขฐาน 10 (Decimal System)

- **สัญลักษณ์ (Digits):** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (รวม 10 ตัว)
- **ฐาน (Base/Radix):** 10
- **แนวคิด:** เป็นระบบที่เราคุ้นเคยและใช้ในชีวิตประจำวันมากที่สุด ค่าของแต่ละหลักจะขึ้นอยู่กับตำแหน่งและเป็นกำลังของ 10
- **ตัวอย่าง:** เลข 10_{10} (สิบฐานสิบ) หมายถึง (1×101)+(0×100)



รูปที่ 2-12 ระบบตัวเลขหลักในคอมพิวเตอร์และการแทนค่าด้วยเลขฐานสอง

2. ระบบเลขฐาน 2 (Binary System)

- **สัญลักษณ์ (Digits):** 0, 1 (รวม 2 ตัว)
- **ฐาน (Base/Radix):** 2
- **แนวคิด:** เป็นภาษาพื้นฐานของคอมพิวเตอร์และอุปกรณ์ดิจิทัลทั้งหมด เนื่องจากสามารถแทนสถานะทางไฟฟ้า 2 สถานะ (เปิด/ปิด, มี/ไม่มีกระแสไฟฟ้า) โดยแต่ละหลักเรียกว่า **บิต (Bit)**
- **ตัวอย่าง:** เลข 1010_2 (หนึ่ง-ศูนย์-หนึ่ง-ศูนย์ ฐานสอง) หมายถึง $(1\times23)+(0\times22)+(1\times21)+(0\times20)=8+0+2+0=10_{10}$
- **ความสำคัญ:** เป็นหัวใจของการทำงานของคอมพิวเตอร์ ข้อมูลทุกอย่างถูกแปลงเป็นเลขฐานสองก่อนประมวลผล

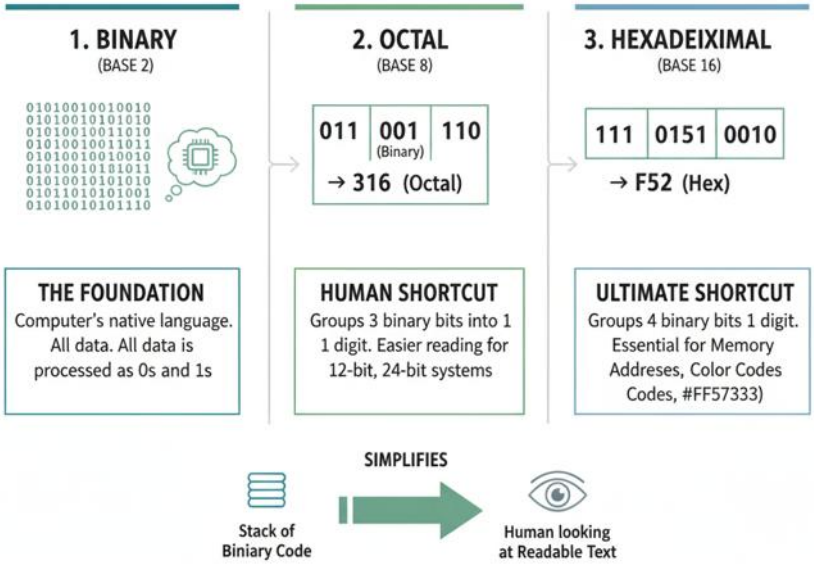
3. ระบบเลขฐาน 8 (Octal System)

- **สัญลักษณ์ (Digits):** 0, 1, 2, 3, 4, 5, 6, 7 (รวม 8 ตัว)
- **ฐาน (Base/Radix):** 8
- **แนวคิด:** ถูกนำมาใช้เพื่อเป็นตัวแทนที่กระชับขึ้นของเลขฐาน 2 เนื่องจาก 23=8 ซึ่งหมายความว่า **เลขฐาน 2 จำนวน 3 บิต สามารถแทนด้วยเลขฐาน 8 เพียง 1 หลัก** ทำให้การอ่านและเขียนเลขฐาน 2 ที่ยาวๆ ง่ายขึ้น
- **ตัวอย่าง:** เลข 12_8 (หนึ่ง-สอง ฐานแปด) หมายถึง $(1\times81)+(2\times80)=8+2=10_{10}$
    - ถ้าเรามีเลขฐานสอง 010 010_2 (ซึ่งคือ 6 บิต) เราสามารถแบ่งเป็นกลุ่มละ 3 บิต (010 010) และแปลงแต่ละกลุ่มเป็นเลขฐาน 8 ได้ $(2\ 2)_8=22_8$ ซึ่งอ่านง่ายกว่า 010010_2

- **การใช้งาน:** เคยนิยมใช้ในระบบคอมพิวเตอร์ยุคแรกๆ ที่มีขนาดคำ (Word Size) เป็นพหุคูณของ 3 บิต (เช่น 12 บิต, 24 บิต)

4. ระบบเลขฐาน 16 (Hexadecimal System)

- **สัญลักษณ์ (Digits):** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F (รวม 16 ตัว โดย A=10, B=11, ..., F=15)
- **ฐาน (Base/Radix):** 16
- **แนวคิด:** เป็นระบบที่ได้รับความนิยมอย่างมากในปัจจุบันสำหรับการทำงานกับคอมพิวเตอร์และระบบดิจิทัล เพราะ 24=16 ซึ่งหมายความว่า **เลขฐาน 2 จำนวน 4 บิต สามารถแทนด้วยเลขฐาน 16 เพียง 1 หลัก** ทำให้การแสดงค่าเลขฐาน 2 ที่ยาวมากๆ (โดยเฉพาะ 8 บิต หรือ 16 บิต) ทำได้กระชับและอ่านง่ายกว่าเลขฐาน 8
- **ตัวอย่าง:** เลข A_{16} (เอ ฐานสิบหก) หมายถึง 10_{10}
    - ถ้าเรามีเลขฐานสอง 1010_2 (ซึ่งคือ 4 บิต) เราสามารถแทนด้วยเลขฐาน 16 เพียง 1 หลัก คือ A_{16}
    - ถ้าเรามีเลขฐานสอง $1010\ 0000_2$ (8 บิต) เราสามารถแบ่งเป็นกลุ่มละ 4 บิต (1010 0000) และแปลงแต่ละกลุ่มเป็นเลขฐาน 16 ได้ (A $0)_{16}$=$A0_{16}$ ซึ่งอ่านง่ายกว่า 10100000_2
- **การใช้งาน:** ใช้กันอย่างแพร่หลายในการแสดงค่าที่อยู่หน่วยความจำ (Memory Address), ค่าสีในงานกราฟิก, รหัสเครื่อง (Machine Code), และในการดีบัก (Debug) โปรแกรมต่างๆ



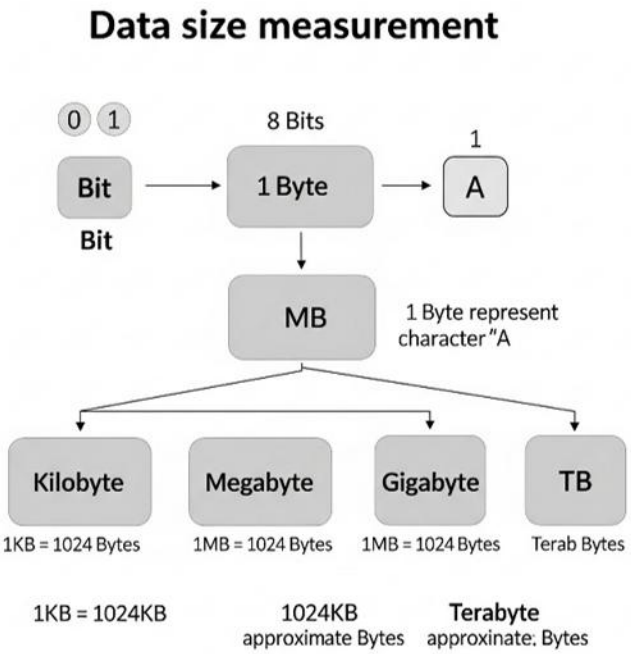
รูปที่ 2-13 การเปรียบเทียบและการลดรูปกลุ่มข้อมูลระหว่างระบบเลขฐานต่าง ๆ

สรุปความสัมพันธ์ระหว่างระบบตัวเลข:

- *เลขฐาน 2 เป็นพื้นฐานที่สุด เพราะเป็นภาษาที่คอมพิวเตอร์เข้าใจโดยตรง*
- *เลขฐาน 8 และ ฐาน 16 เป็นระบบที่ถูกสร้างขึ้นมาเพื่ออำนวยความสะดวกให้มนุษย์ในการทำงานกับเลขฐาน 2 จำนวนมากๆ โดยทำหน้าที่เป็น "ตัวย่อ" หรือ "ตัวแทน" ของกลุ่มบิตฐาน 2 ทำให้การอ่าน, เขียน, และทำความเข้าใจข้อมูลในระดับต่ำ (Low-level) ของคอมพิวเตอร์ง่ายขึ้น*

5.การวัดขนาดข้อมูล (Data Size Measurement): บิต (Bit) และ ไบต์ (Byte)

1. บิต (Bit)นิยาม: บิต (Bit) ย่อมาจากคำว่า **Bi**nary digi**t** (ตัวเลขฐานสอง) เป็นหน่วยข้อมูลที่เล็กที่สุดในระบบคอมพิวเตอร์



รูปที่ 2-14 หน่วยวัดขนาดของข้อมูลและการจัดลำดับชั้นความจุ (Data Size Measurement)

ค่าที่เก็บได้: 1 บิตสามารถเก็บค่าได้เพียง 2 สถานะเท่านั้น คือ **0** หรือ **1**

เปรียบเสมือนสวิตช์ไฟ 1 ดวง ที่มีแค่สถานะ เปิด หรือ ปิด

ความสำคัญ: เป็นรากฐานของข้อมูลทั้งหมดในคอมพิวเตอร์ ไม่ว่าข้อมูลจะซับซ้อนแค่ไหน สุดท้ายแล้วก็จะถูกแปลงเป็นชุดของบิต 0 และ 1

2. ไบต์ (Byte)

นิยาม: ไบต์ (Byte) คือกลุ่มของบิตที่รวมกันเพื่อแทนข้อมูลที่มีความหมาย

- **ค่ามาตรฐาน:** โดยทั่วไป 1 ไบต์ มีค่าเท่ากับ **8 บิต**
    - เมื่อเรานำ 8 บิตมารวมกัน แต่ละบิตสามารถเป็น 0 หรือ 1 ได้ ทำให้ 1 ไบต์สามารถสร้างรูปแบบที่แตกต่างกันได้ถึง 28=256 รูปแบบ
- **การใช้งาน:**
    - **การแทนตัวอักษร:** 1 ไบต์มักใช้ในการแทนตัวอักษร 1 ตัว (เช่น A, B, ก, ข) หรือสัญลักษณ์พิเศษ 1 ตัว ตามมาตรฐานรหัสตัวอักษรต่างๆ เช่น ASCII
    - **การเก็บข้อมูล:** เป็นหน่วยพื้นฐานที่ใช้ในการวัดขนาดของไฟล์ข้อมูลต่างๆ เช่น เอกสาร รูปภาพ หรือไฟล์เสียง
- **ตัวอย่าง:**
    - ตัวอักษร 'A' ในรหัส ASCII จะถูกแทนด้วยเลขฐานสอง 01000001 ซึ่งมีขนาด 8 บิต หรือ 1 ไบต์
    - ตัวอักษร 'ก' อาจใช้ 2 ไบต์ในระบบ Unicode

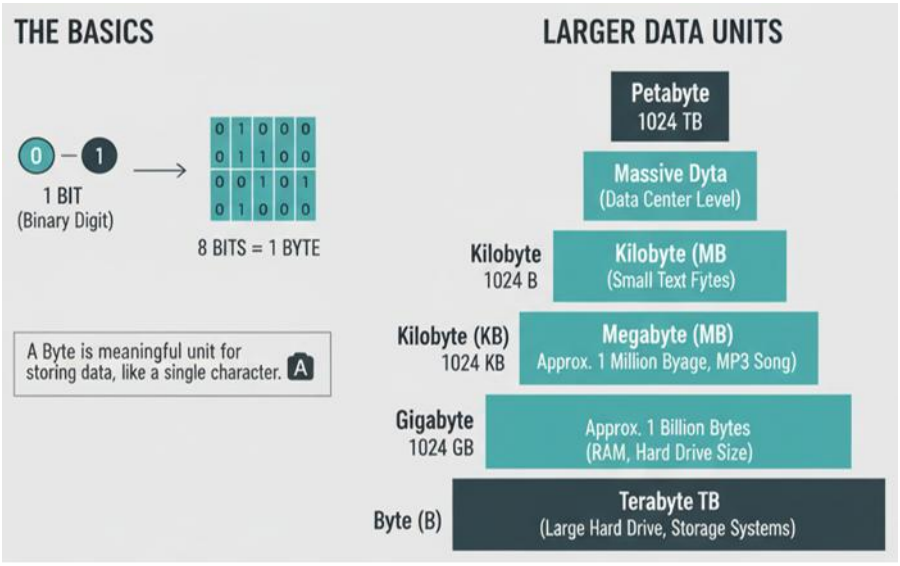
ความสัมพันธ์ระหว่าง บิต และ ไบต์

ไบต์เป็นหน่วยที่ใหญ่กว่าบิต และถูกสร้างขึ้นจากบิตหลายๆ บิตรวมกันเพื่อวัตถุประสงค์ในการจัดเก็บและประมวลผลข้อมูลที่มีความหมาย

- 1 บิต (bit) = 0 หรือ 1 และ 8 บิต (bits) = 1 ไบต์ (Byte)

5.1 หน่วยวัดขนาดข้อมูลที่ใหญ่ขึ้น

เพื่อที่จะอธิบายขนาดของข้อมูลที่ใหญ่ขึ้น เราจะใช้คำอุปสรรค (Prefix) มาต่อท้ายคำว่า "ไบต์" คล้ายกับการวัดน้ำหนัก (กรัม, กิโลกรัม) หรือระยะทาง (เมตร, กิโลเมตร)



รูปที่ 2-15 พีระมิดแสดงลำดับหน่วยความจุข้อมูลและการเปรียบเทียบการใช้งานจริง

หน่วย	ขนาดเทียบกับไบต์	ค่าโดยประมาณ (สำหรับผู้ใช้งาน)
กิโลไบต์ (KB)	1 KB=2^{10} bytes=1024 bytes	ประมาณ 1 พันไบต์ (ใช้สำหรับไฟล์เอกสารขนาดเล็ก)
เมกะไบต์ (MB)	1 MB=2^{20} bytes=1024 KB	ประมาณ 1 ล้านไบต์ (ใช้สำหรับไฟล์รูปภาพ, เพลง MP3)
กิกะไบต์ (GB)	1 GB=2^{30} bytes=1024 MB	ประมาณ 1 พันล้านไบต์ (ใช้สำหรับขนาดหน่วยความจำ RAM, ฮาร์ดดิสก์)
เทราไบต์ (TB)	1 TB=2^{40} bytes=1024 GB	ประมาณ 1 ล้านล้านไบต์ (ใช้สำหรับขนาดฮาร์ดดิสก์ขนาดใหญ่, ระบบจัดเก็บข้อมูล)
เพตาไบต์ (PB)	1 PB=2^{50} bytes=1024 TB	ใช้สำหรับข้อมูลขนาดใหญ่มากในระดับศูนย์ข้อมูล (Data Center)

ตารางที่ 2-1 การเปรียบเทียบหน่วยวัดขนาดข้อมูลและตัวอย่างการใช้งานในระบบคอมพิวเตอร์

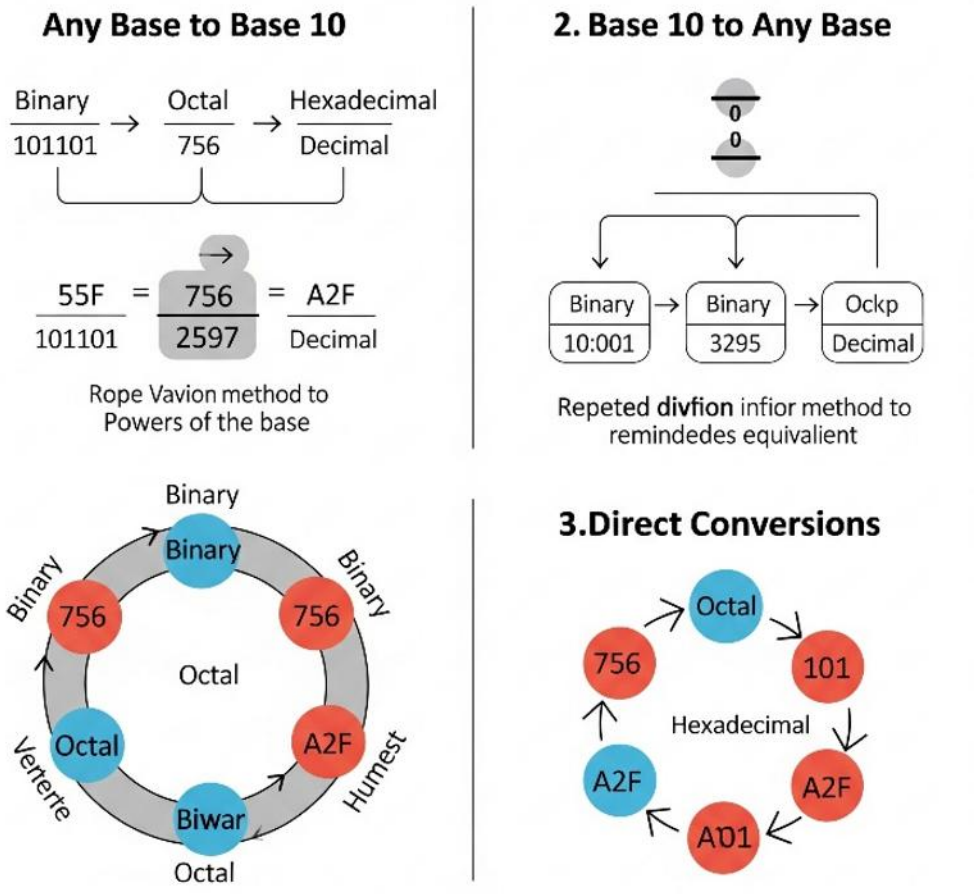
สรุป ทำไมถึงใช้ 1024 แทน 1000?

ในทางคอมพิวเตอร์นิยมใช้ 2^{10}=1024 เนื่องจากคอมพิวเตอร์ทำงานในระบบเลขฐาน 2 อย่างไรก็ตาม ในทางการตลาดหรือการโฆษณาอุปกรณ์จัดเก็บข้อมูล อาจมีการใช้ 1000 เป็นตัวคูณเพื่อให้ตัวเลข

ขนาดความจุดูสูงขึ้น (เช่น ฮาร์ดดิสก์ 1 TB จริงๆ อาจมีพื้นที่ใช้งานประมาณ 0.909 TB หรือ 909 GB โดยประมาณเมื่อวัดด้วย 1024 bytes/KB)

6.การแปลงเลขฐาน (Base Conversion)

การแปลงเลขฐานคือกระบวนการเปลี่ยนการนำเสนอค่าตัวเลขจากระบบฐานหนึ่งไปเป็นอีกระบบฐานหนึ่ง คอมพิวเตอร์จำเป็นต้องมีการแปลงเลขฐานอยู่เสมอ เพื่อให้สามารถสื่อสารและประมวลผลข้อมูลในรูปแบบที่เหมาะสม



รูปที่ 2-16 สรุปวิธีการและขั้นตอนการแปลงระบบเลขฐานประเภทต่าง ๆ

6.1. การแปลงเลขฐานใดๆ (Binary, Octal, Hexadecimal) ไปเป็น เลขฐาน 10 (Decimal)

วิธีนี้เป็นวิธีที่ง่ายที่สุดและเป็นพื้นฐานของการแปลงเลขฐานอื่นๆ หลักการคือการใช้ **ค่าประจำหลัก (Place Value)** ของแต่ละตัวเลขในระบบฐานนั้นๆ คูณกับตัวเลขโดด แล้วนำผลลัพธ์ทั้งหมดมารวมกัน

หลักการ:

1. กำหนดตำแหน่งของหลัก โดยเริ่มจาก 0 ที่หลักขวาสุด แล้วเพิ่มขึ้นไปทางซ้าย (1, 2, 3, ...)
2. สำหรับแต่ละหลัก ให้เอา "ตัวเลขโดด" ในหลักนั้นๆ คูณกับ "ฐาน" ยกกำลังด้วย "ตำแหน่งของหลัก"

3. นำผลลัพธ์ที่ได้จากการคูณในแต่ละหลักมารวมกันทั้งหมด

สูตรทั่วไป: ถ้ามีเลขฐาน R (เช่น R=2, 8, 16) เป็น $d_n d_{n-1} \ldots d_1 d_0 . d_{-1} d_{-2} \ldots d_{-m}$ จะมีค่าเท่ากับ:

$(d_n \times R^n) + (d_{n-1} \times R^{n-1}) + \ldots + (d_1 \times R^1) + (d_0 \times R^0) + (d_{-1} \times R^{-1}) + \ldots + (d_{-m} \times R^{-m})$

ตัวอย่างที่ 1: การแปลงเลขฐาน 2 (Binary) เป็น เลขฐาน 10 (Decimal)

โจทย์: แปลง 1011_2 ให้เป็นเลขฐาน 10

วิธีทำ:

ขั้นที่ 1: กำหนดตำแหน่งหลัก

- $1\ 0\ 1\ 1_2$

ตำแหน่ง: 3 2 1 0

ขั้นที่ 2: คูณตัวเลขโดดด้วยฐานยกกำลังตำแหน่งหลัก

- $(1\times2^3)+(0\times2^2)+(1\times2^1)+(1\times2^0)$

ขั้นที่ 3: คำนวณผลลัพธ์และรวมกัน

- $=(1\times8)+(0\times4)+(1\times2)+(1\times1)$
- $=8+0+2+1$
- $=11_{10}$

ดังนั้น $1011_2=11_{10}$

ตัวอย่างที่ 2: การแปลงเลขฐาน 8 (Octal) เป็น เลขฐาน 10 (Decimal)

โจทย์: แปลง 237_8 ให้เป็นเลขฐาน 10

วิธีทำ:

ขั้นที่ 1: กำหนดตำแหน่งหลัก

- 237_8

ตำแหน่ง: 2 1 0

ขั้นที่ 2: คูณตัวเลขโดดด้วยฐานยกกำลังตำแหน่งหลัก

- $(2\times8^2)+(3\times8^1)+(7\times8^0)$

ขั้นที่ 3: คำนวณผลลัพธ์และรวมกัน

- $=(2\times64)+(3\times8)+(7\times1)$
- $=128+24+7 =159_{10}$

ดังนั้น $237_8=159_{10}$

ตัวอย่างที่ 3: การแปลงเลขฐาน 16 (Hexadecimal) เป็น เลขฐาน 10 (Decimal)

โจทย์: แปลง $A5_{16}$ ให้เป็นเลขฐาน 10 (จำไว้ว่า A = 10)

วิธีทำ:

ขั้นที่ 1: กำหนดตำแหน่งหลัก

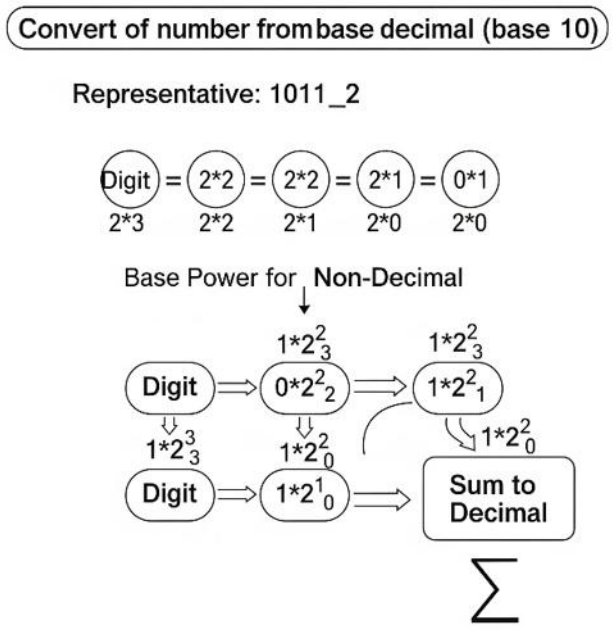
- $A5_{16}$

ตำแหน่ง: 1 0

ขั้นที่ 2: คูณตัวเลขโดดด้วยฐานยกกำลังตำแหน่งหลัก (แทน A ด้วย 10)

- $(A\times16^1) + (5\times16^0)$
- $=(10\times16^1)+(5\times16^0)$
- **ขั้นที่ 3: คำนวณผลลัพธ์และรวมกัน**
- $=(10\times16)+(5\times1)$
- $=160+5$
- $=165_{10}$

ดังนั้น $A5_{16}=165_{10}$



รูปที่ 2-17 ขั้นตอนการแปลงเลขฐานสองเป็นเลขฐานสิบด้วยวิธีค่าประจำหลัก (Positional Notation)

สรุป จากภาพแสดงให้เห็นถึงหลักการพื้นฐานในการแปลงเลขฐานใดๆ (เช่น ฐาน 2, 8, 16) ให้เป็นเลขฐาน 10 (Decimal) ซึ่งเป็นวิธีที่ใช้การคูณและบวก:

- ***เลขฐาน (Base Number):*** *ตัวเลขที่เราต้องการแปลง (ในภาพคือ 1011_2)*
- ***Digits (ตัวเลขโดด):*** *แต่ละตัวเลขในตำแหน่งต่างๆ ของเลขฐานนั้นๆ (ในภาพคือ 1, 0, 1, 1)*
- ***Base Power (ฐานยกกำลังตำแหน่ง):*** *แต่ละหลักจะมีค่าประจำหลักเป็น "ฐาน" ยกกำลังด้วย "ตำแหน่งของหลัก" โดยเริ่มจากตำแหน่ง 0 ที่หลักขวาสุด แล้วเพิ่มขึ้นไปทางซ้าย (เช่น $2^3,2^2,2^1,2^0$ สำหรับเลขฐาน 2)*
- ***Sum to Decimal (ผลรวมเป็นเลขฐาน 10):*** *นำผลคูณของ "ตัวเลขโดด" กับ "ฐานยกกำลังตำแหน่ง" ของแต่ละหลักมารวมกันทั้งหมด ก็จะได้ค่าในระบบเลขฐาน 10*

6.2. การแปลงเลขฐาน 10 (Decimal) ไปเป็น เลขฐานใดๆ (Binary, Octal, Hexadecimal)

วิธีนี้จะใช้หลักการ **การหารด้วยฐานของระบบเลขที่ต้องการแปลงไปหา** และเก็บเศษที่ได้จากการหาร โดยจะทำซ้ำไปเรื่อยๆ จนกระทั่งผลหารเป็นศูนย์ จากนั้นนำเศษที่ได้มาเรียงจากล่างขึ้นบน (จากเศษตัวสุดท้ายไปเศษตัวแรก)

หลักการ:

1. นำเลขฐาน 10 ที่ต้องการแปลงไปหารด้วย "ฐาน" ของระบบเลขที่ต้องการแปลงไปหา
2. จด "เศษ" ที่ได้จากการหาร
3. นำ "ผลหาร" ที่ได้ในข้อ 1 มาเป็นตัวตั้งในการหารครั้งต่อไป
4. ทำซ้ำขั้นตอนที่ 1-3 จนกระทั่งผลหารเป็นศูนย์
5. นำเศษที่จดไว้ทั้งหมดมาเรียงจากล่างขึ้นบน (จากเศษตัวสุดท้ายไปเศษตัวแรก)

ตัวอย่างที่ 4: การแปลงเลขฐาน 10 (Decimal) เป็น เลขฐาน 2 (Binary)

โจทย์: แปลง 11_{10} ให้เป็นเลขฐาน 2

วิธีทำ: (หารด้วย 2)

- 11÷2=5 เศษ 1
- 5÷2=2 เศษ 1
- 2÷2=1 เศษ 0
- 1÷2=0 เศษ 1

นำเศษที่ได้มาเรียงจากล่างขึ้นบน: 1011

ดังนั้น $11_{10}=1011_2$

ตัวอย่างที่ 5: การแปลงเลขฐาน 10 (Decimal) เป็น เลขฐาน 8 (Octal)

โจทย์: แปลง 159_{10} ให้เป็นเลขฐาน 8

วิธีทำ: (หารด้วย 8)

- 159÷8=19 เศษ 7
- 19÷8=2 เศษ 3
- 2÷8=0 เศษ 2

นำเศษที่ได้มาเรียงจากล่างขึ้นบน: 237

ดังนั้น $159_{10}=237_8$

ตัวอย่างที่ 6: การแปลงเลขฐาน 10 (Decimal) เป็น เลขฐาน 16 (Hexadecimal)

โจทย์: แปลง 165_{10} ให้เป็นเลขฐาน 16

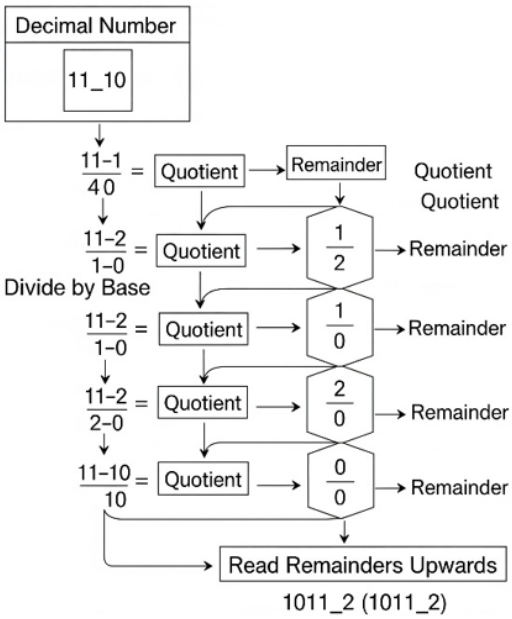
วิธีทำ: (หารด้วย 16)

- 165÷16=10 เศษ 5 (จำไว้ว่า 10 ในฐาน 16 คือ A)
- 10÷16=0 เศษ 10 (ซึ่งคือ A)

นำเศษที่ได้มาเรียงจากล่างขึ้นบน: A5

ดังนั้น 165_{10}=$A5_{16}$

จากภาพแสดงถึงวิธีการแปลงเลขฐาน 10 (Decimal) ไปเป็นเลขฐานอื่นๆ (เช่น ฐาน 2, 8, 16) โดยใช้หลักการ **การหารด้วยฐาน** และการเก็บเศษ:

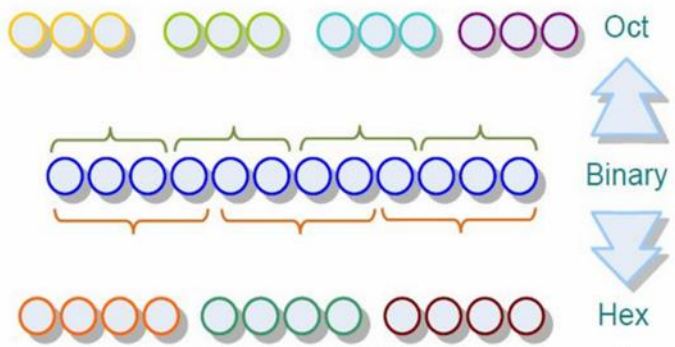


รูปที่ 2-18 ขั้นตอนการแปลงเลขฐานสิบเป็นเลขฐานสองด้วยวิธีหารซ้ำ (Repeated Division Method)

- **Decimal Number (เลขฐาน 10):** ตัวเลขเริ่มต้นที่เราต้องการแปลง (ในภาพคือ 11_{10})
- **Divide by Base (หารด้วยฐาน):** นำเลขฐาน 10 มาหารด้วย "ฐาน" ของระบบเลขที่ต้องการแปลงไปหา (เช่น หารด้วย 2 สำหรับฐาน 2)
- **Quotient (ผลหาร):** ผลลัพธ์ของการหารในแต่ละครั้ง
- **Remainder (เศษ):** เศษที่ได้จากการหารในแต่ละครั้ง ซึ่งจะเป็นตัวเลขโดดในระบบฐานใหม่
- **Read Remainders Upwards (อ่านเศษจากล่างขึ้นบน):** ทำการหารซ้ำไปเรื่อยๆ จนกระทั่งผลหารเป็น 0 จากนั้นนำเศษที่ได้ในแต่ละขั้นตอนมาเรียงต่อกันจากล่างขึ้นบน ก็จะได้ค่าในระบบเลขฐานใหม่

6.3. การแปลงเลขฐาน 2 (Binary), ฐาน 8 (Octal), และ ฐาน 16 (Hexadecimal) โดยตรง

การแปลงระหว่างเลขฐาน 2, 8, และ 16 สามารถทำได้โดยตรงโดยไม่ต้องผ่านเลขฐาน 10 เนื่องจากมีความสัมพันธ์กันในเชิงกำลังของ 2 (2^1=2, 2^3=8, 2^4=16)



รูปที่ 2-19 วิธีการจัดกลุ่มบิตเพื่อแปลงเลขฐานสองเป็นเลขฐานแปดและฐานสิบหก

หลักการสำคัญ:

1 หลักฐาน 8 = 3 บิตฐาน 2 (เพราะ 2^3=8)

1 หลักฐาน 16 = 4 บิตฐาน 2 (เพราะ 2^4=16)

6.3.1 การแปลงเลขฐาน 2 (Binary) เป็น เลขฐาน 8 (Octal)

หลักการ: แบ่งกลุ่มเลขฐาน 2 ออกเป็นกลุ่มละ 3 บิต โดยเริ่มจากขวาไปซ้าย (ถ้าบิตไม่พอให้เติม 0 ด้านหน้า) แล้วแปลงแต่ละกลุ่ม 3 บิตนั้นให้เป็นเลขฐาน 8

โจทย์: แปลง 1011011_2 เป็นเลขฐาน 8

วิธีทำ:

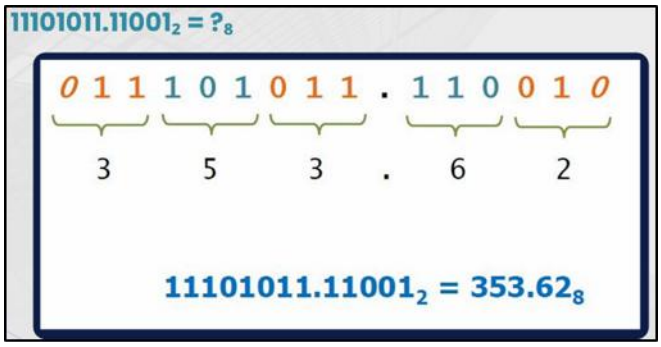
1. **แบ่งกลุ่ม 3 บิตจากขวาไปซ้าย:**

   1 011 011 **(เติม 0 ด้านหน้ากลุ่มแรกเพื่อให้ครบ 3 บิต:** 001 011 011**)**

2. **แปลงแต่ละกลุ่มเป็นเลขฐาน 8:**
   - $001_2 = (0 * 2^2) + (0 * 2^1) + (1 * 2^0) = 0 + 0 + 1 = 1_8$
   - $011_2 = (0 * 2^2) + (1 * 2^1) + (1 * 2^0) = 0 + 2 + 1 = 3_8$
   - $011_2 = (0 * 2^2) + (1 * 2^1) + (1 * 2^0) = 0 + 2 + 1 = 3_8$
3. **นำผลลัพธ์มารวมกัน:** 133_8

ดังนั้น $1011011_2=133_8$

กรณีถ้าเป็นจุดทศนิยม



รูปที่ 2-20 ตัวอย่างการแปลงเลขฐานสองที่มีทศนิยมเป็นเลขฐานแปดโดยการจัดกลุ่ม 3 บิต

6.3.2 การแปลงเลขฐาน 8 (Octal) เป็น เลขฐาน 2 (Binary)

หลักการ: แปลงแต่ละหลักของเลขฐาน 8 ให้เป็นเลขฐาน 2 จำนวน 3 บิต (ถ้าได้ไม่ครบ 3 บิตให้เติม 0 ด้านหน้า) แล้วนำบิตทั้งหมดมารวมกัน

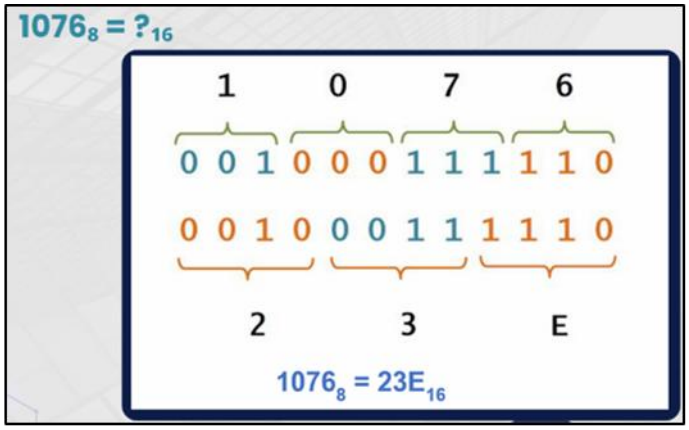
โจทย์: แปลง 133_8 เป็นเลขฐาน 2

วิธีทำ:

1. **แปลงแต่ละหลักเป็นเลขฐาน 2 จำนวน 3 บิต:**
   - $1_8 = 001_2$ $3_8 = 011_2$ $3_8 = 011_2$
2. **นำบิตทั้งหมดมารวมกัน:** 001011011_2 (สามารถตัด 0 ที่อยู่หน้าสุดออกได้)

ดังนั้น $133_8=1011011_2$

การแปลงเลขฐาน 8 เป็นฐาน 16



รูปที่ 2-21 ตัวอย่างการแปลงเลขฐานแปดเป็นเลขฐานสิบหก โดยใช้เลขฐานสองเป็นตัวกลาง

6.3.3 การแปลงเลขฐาน 2 (Binary) เป็น เลขฐาน 16 (Hexadecimal)

หลักการ: แบ่งกลุ่มเลขฐาน 2 ออกเป็นกลุ่มละ 4 บิต โดยเริ่มจากขวาไปซ้าย (ถ้าบิตไม่พอให้เติม 0 ด้านหน้า) แล้วแปลงแต่ละกลุ่ม 4 บิตนั้นให้เป็นเลขฐาน 16

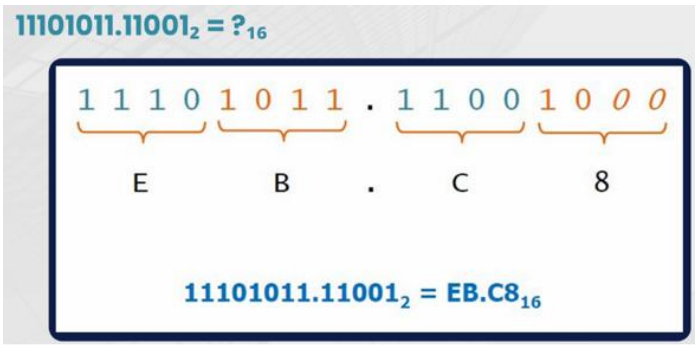
โจทย์: แปลง 10110110_2 เป็นเลขฐาน 16

วิธีทำ:

1. **แบ่งกลุ่ม 4 บิตจากขวาไปซ้าย:** 1011 0110
2. **แปลงแต่ละกลุ่มเป็นเลขฐาน 16:**
   - $1011_2 = (1 * 2^{\wedge 3}) + (0 * 2^{\wedge 2}) + (1 * 2^{\wedge 1}) + (1 * 2^{\wedge 0})$
     $= 8 + 0 + 2 + 1$
     $= 11_{10}$ **หรือ** $= B_{16}$
   - $0110_2 = (0 * 2^{\wedge 3}) + (1 * 2^{\wedge 2}) + (1 * 2^{\wedge 1}) + (0 * 2^{\wedge 0})$
     $= 0 + 4 + 2 + 0$
     $= 6_{10}$ **หรือ** $= 6_{16}$
3. **นำผลลัพธ์มารวมกัน:** $B6_{16}$

ดังนั้น $10110110_2 = B6_{16}$

ตัวอย่าง กรณีเป็นทศนิยม



รูปที่ 2-22 ตัวอย่างการแปลงเลขฐานสองที่มีทศนิยมเป็นเลขฐานสิบหกโดยการจัดกลุ่ม 4 บิต

6.3.4 การแปลงเลขฐาน 16 (Hexadecimal) เป็น เลขฐาน 2 (Binary)

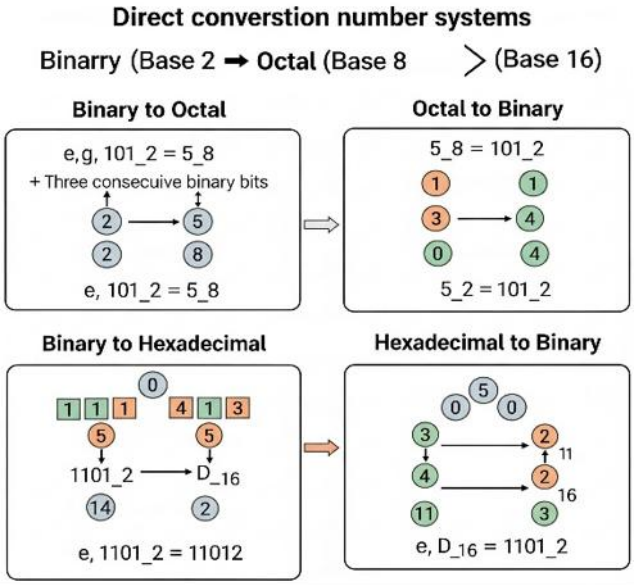
หลักการ: แปลงแต่ละหลักของเลขฐาน 16 ให้เป็นเลขฐาน 2 จำนวน 4 บิต (ถ้าได้ไม่ครบ 4 บิตให้เติม 0 ด้านหน้า) แล้วนำบิตทั้งหมดมารวมกัน

โจทย์: แปลง $B6_{16}$ เป็นเลขฐาน 2 **วิธีทำคือ :**

1. **แปลงแต่ละหลักเป็นเลขฐาน 2 จำนวน 4 บิต:**
   - B_{16} (ซึ่งคือ 11_{10}) $=1011_2$
   - 6_{16} (ซึ่งคือ 6_{10}) $=0110_2$
2. **นำบิตทั้งหมดมารวมกัน:** 10110110_2

ดังนั้น $B6_{16}=10110110_2$

สรุปจากภาพแสดงความสัมพันธ์โดยตรงและวิธีการแปลงระหว่างเลขฐาน 2 (Binary), ฐาน 8 (Octal), และฐาน 16 (Hexadecimal) เป็นวิธีที่รวดเร็วและไม่ต้องผ่านเลขฐาน 10



รูปที่ 2-23 แผนผังการแปลงเลขฐานสลับไปมาโดยตรง (Direct Conversion Number Systems)

- **Binary to Octal (ฐาน 2 เป็น ฐาน 8):**
  - **หลักการ:** แบ่งกลุ่มเลขฐาน 2 ออกเป็นกลุ่มละ **3 บิต** โดยเริ่มจากขวาไปซ้าย (ถ้าไม่ครบ 3 บิต ให้เติม 0 ข้างหน้า)
  - **ตัวอย่าง:** 101_2 จะเท่ากับ 5_8
- **Octal to Binary (ฐาน 8 เป็น ฐาน 2):**
  - **หลักการ:** แปลงแต่ละหลักของเลขฐาน 8 ให้เป็นเลขฐาน 2 จำนวน **3 บิต** (ถ้าได้ไม่ครบ 3 บิต ให้เติม 0 ข้างหน้า)
  - **ตัวอย่าง:** 5_8 จะเท่ากับ 101_2
- **Binary to Hexadecimal (ฐาน 2 เป็น ฐาน 16):**
  - **หลักการ:** แบ่งกลุ่มเลขฐาน 2 ออกเป็นกลุ่มละ **4 บิต** โดยเริ่มจากขวาไปซ้าย (ถ้าไม่ครบ 4 บิต ให้เติม 0 ข้างหน้า)
  - **ตัวอย่าง:** 1101_2 จะเท่ากับ D_{16} (โดยที่ D แทนค่า 13_{10})
- **Hexadecimal to Binary (ฐาน 16 เป็น ฐาน 2):**
  - **หลักการ:** แปลงแต่ละหลักของเลขฐาน 16 ให้เป็นเลขฐาน 2 จำนวน **4 บิต** (ถ้าได้ไม่ครบ 4 บิต ให้เติม 0 ข้างหน้า)
  - **ตัวอย่าง:** D_{16} จะเท่ากับ 1101_2

7.การคำนวณทางคณิตศาสตร์ในระบบคอมพิวเตอร์ (Arithmetic Operations)

การคำนวณทางคณิตศาสตร์ในระบบคอมพิวเตอร์ถือเป็นหัวใจหลักของหน่วยประมวลผลทางคณิตศาสตร์และตรรกะ (ALU - Arithmetic Logic Unit) เลยนะครับ คอมพิวเตอร์ไม่ได้คำนวณแบบที่เราทำในเลขฐาน 10 แต่จะคำนวณด้วยเลขฐาน 2 เป็นหลัก รวมถึงมีวิธีการพิเศษในการลบเลขด้วย 1's Complement และ 2's Complement เพื่อทำให้การออกแบบวงจรทำได้ง่ายขึ้น

7.1. การบวกเลขฐานสอง (Binary Addition)

การบวกเลขฐานสองมีหลักการคล้ายกับการบวกเลขฐาน 10 แต่มีกฎง่ายๆ ดังนี้

กฎการบวกเลขฐานสอง:

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1= 0 ทด **1** (Carry 1)
- 1 + 1 + 1 = 1 ทด **1** (Carry 1) (กรณีมีตัวทดจากหลักก่อนหน้า)

ตัวอย่าง: บวก 1011_2+110_2

```
  1 1  <--    ตัวทด (Carry)
  1011₂       (เท่ากับ 11 ในฐาน 10)
+ 0110₂       (เท่ากับ 6 ในฐาน 10, เติม 0 ด้านหน้าให้มีจำนวนบิตเท่ากัน)
---------------
 10001₂       (เท่ากับ 17 ในฐาน 10)
```

อธิบายทีละหลัก (จากขวาไปซ้าย):

- หลักที่ 0 (บิตขวาสุด): 1+0=1
- หลักที่ 1: 1+1=0 ทด 1 ไปหลักถัดไป
- หลักที่ 2: 0+1+1(ทด)=0 ทด 1 ไปหลักถัดไป
- หลักที่ 3: 1+0+1(ทด)=0 ทด 1 ไปหลักถัดไป
- หลักที่ 4 (หลักใหม่จากการทด): 0+0+1(ทด)=1

ผลลัพธ์คือ 10001_2

7.2. การลบเลขฐานสอง (Binary Subtraction)

การลบเลขฐานสองก็คล้ายกับการลบเลขฐาน 10 แต่มีการ "ยืม" เกิดขึ้นเมื่อตัวตั้งน้อยกว่าตัวลบในหลักนั้นๆ

กฎการลบเลขฐานสอง:

- 0 – 0 = 0
- 1 – 0 = 1
- 1 – 1 = 0
- 0 – 1 = 1 และยืม **1** จากหลักถัดไปทางซ้าย (เมื่อยืมมา ค่าของหลักที่ยืมมาจะลดลง 1 ส่วนหลักที่ยืมไปจะกลายเป็น 2)

ตัวอย่าง: ลบ $1011_2 - 110_2$

0 10 **<-- ยืมจากหลักถัดไป (Borrow)**

1 0 1 1_2 **(เท่ากับ 11 ในฐาน 10)**

- 0 1 1 0_2 **(เท่ากับ 6 ในฐาน 10)**

---

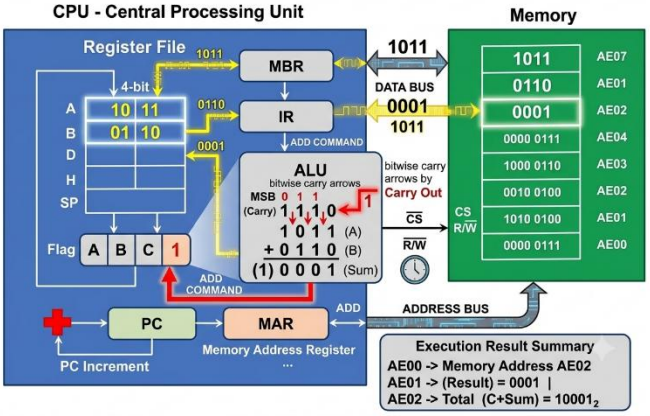
0 1 0 1_2 (เท่ากับ 5 ในฐาน 10)

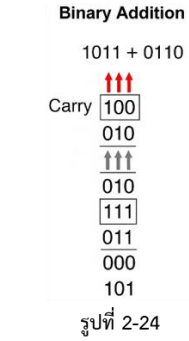
อธิบายทีละหลัก (จากขวาไปซ้าย):

- **หลักที่ 0:** 1−0=1
- **หลักที่ 1:** 1−1=0
- **หลักที่ 2:** 0−1 ไม่ได้ ต้องยืมจากหลักที่ 3 (ซึ่งคือ 1)
  - เมื่อยืมมา 1 จากหลักที่ 3 หลักที่ 3 จะกลายเป็น 0
  - หลักที่ 2 จะกลายเป็น 10_2 (ซึ่งคือ 2 ในฐาน 10) ดังนั้น $10_2 - 1_2 = 1_2$
- **หลักที่ 3:** (หลังจากถูกยืมไป) กลายเป็น 0−0=0

ผลลัพธ์คือ 0101_2 **หรือ** 101_2

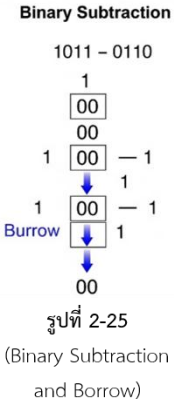
สรุปวิธีการบวกและลบเลขฐานสอง ซึ่งเป็นพื้นฐานการคำนวณของคอมพิวเตอร์:





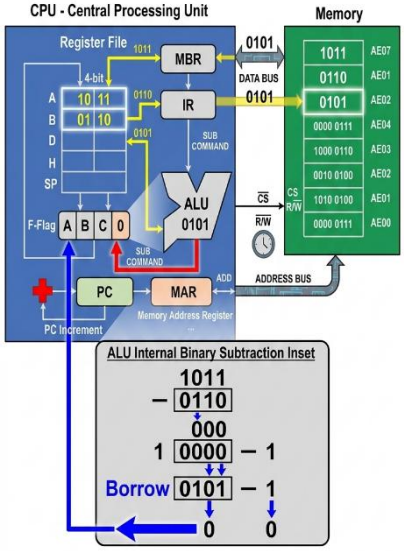
รูปที่ 2-24
(Binary Addition and Carry)

- *Binary Addition (การบวกเลขฐานสอง):*
  - *แสดงการบวก 1011_2 กับ 0110_2*
  - *ลูกศรชี้ขึ้น (สีแดง) แสดงถึง **ตัวทด (Carry)** ที่เกิดขึ้นเมื่อผลบวกเป็น 2 (หรือมากกว่า) ซึ่งจะถูกทดไปยังหลักถัดไปทางซ้าย และ ผลลัพธ์ที่ได้คือ 10001_2*



รูปที่ 2-25
(Binary Subtraction and Borrow)

- *Binary Subtraction (การลบเลขฐานสอง):*
  - *แสดงการลบ 1011_2 ด้วย 0110_2*
  - *ลูกศรชี้ลง (สีน้ำเงิน) แสดงถึง **การยืม (Borrow)** ที่เกิดขึ้นเมื่อตัวตั้งในหลักนั้นน้อยกว่าตัวลบ ทำให้ต้องยืม 1 จากหลักถัดไปทางซ้าย (ซึ่ง 1 ที่ยืมมาในฐาน 2 จะมีค่าเท่ากับ 2)*
  - *ผลลัพธ์ที่ได้คือ 0101_2*



7.3 การบวกเลขฐาน 8 (Octal Addition) หลักการคือ

1. ความแตกต่างของเลข "ฐาน"

- **เลขฐาน 10 (Decimal):** ใช้อยู่ทุกวัน มีตัวเลขโดด 10 ตัว คือ 0 ถึง 9 เมื่อผลบวกเท่ากับ **10** หรือมากกว่า ต้อง **ทด 1** ไปหลักถัดไป (เพราะ $10_{10} = 10_2$ ในฐาน 10 คือ 1 หน่วยของหลักสิบ กับ 0 หน่วยของหลักหน่วย)
- **เลขฐาน 8 (Octal):** มีตัวเลขโดด 8 ตัว คือ **0 ถึง 7** เมื่อผลบวกเท่ากับ **8** หรือมากกว่า ต้อง **ทด 1** ไปหลักถัดไป (เพราะ $8_{10} = 10_8$ ในฐาน 8 คือ 1 หน่วยของหลักแปด กับ 0 หน่วยของหลักหน่วย)

คุณสมบัติ	เลขฐาน 10 (Decimal)	เลขฐาน 8 (Octal)
ตัวเลขโดดที่ใช้	10 ตัวคือ: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9	8 ตัวคือ: **0, 1, 2, 3, 4, 5, 6, 7**
จุดเปลี่ยนการทด (Carry Point)	เมื่อผลบวก ≥ 10	เมื่อผลบวก ≥ 8
ความหมายของ "10"	10_{10} (หนึ่งหลักสิบ)	10_8 (หนึ่งหลักแปด)
กฎการทด (Carry Rule)	ถ้าผลบวกครบ **10** หน่วย **ทด 1** ไปหลักถัดไป	ถ้าผลบวกครบ **8** หน่วย **ทด 1** ไปหลักถัดไป

ตารางที่ 2-2 กฎพื้นฐานสำหรับการบวกและการลบในระบบเลขฐานสอง

2. กฎสำคัญที่สุด: "8 คือ 10 ในฐาน 8"

ถ้าผลบวกในหลักใด ๆ ได้ **8** ตัวเลขจะ "รีเซ็ต" เป็น **0** แล้วมี **1** ถูก "ส่งต่อ" ไปหลักข้างหน้า เปรียบเหมือนนาฬิกาที่พอถึงเลข 8 (หรือ 60 นาทีในนาฬิกา) ก็จะกลับไปที่ 0 แล้วส่งไปเป็น 1 ชั่วโมงข้างหน้า

การเปรียบเทียบกับนาฬิกา:

เปรียบเหมือนการนับเวลาในฐาน 8 ที่มีแค่ 0 ถึง 7 พอถึง หน่วย (ครบฐาน) จะทำให้ตัวเลขจะรีเซ็ตเป็น 0 แล้ว ส่ง 1 (ตัวทด) ไปหลักข้างหน้า

ตัวอย่างการบวก (ในหลักปัจจุบัน)	ผลบวก (ฐาน 10)	การคำนวณในฐาน 8	ผลลัพธ์ในฐาน 8
$3_8 + 4_8$	7	7 < 8 ทำให้ **ไม่ทด**	7_8 (ทด 0)
$5_8 + 3_8$	8	8 ÷ 8 = 1 เศษ 0	10_8 (เขียน 0, ทด **1**)
$6_8 + 7_8$$	13	13 ÷ 8 = 1 เศษ 5	15_8 (เขียน 5, ทด **1**)

ตารางที่ 2-3 ตัวอย่างการคำนวณผลบวกและการทดเลขในระบบเลขฐานแปด

3. การทำความเข้าใจ "กฎการทด" อย่างละเอียด

นี่คือส่วนที่สำคัญที่สุดและเป็นหัวใจของการบวกเลขฐานใด ๆ ที่ไม่ใช่ฐาน 10

แนวคิด: การหารเอาเศษ (Modulo and Floor)

เมื่อเราบวกตัวเลขในหลักหนึ่งแล้วได้ผลบวกที่เกิน 7 (เช่น ได้ 8, 9, 10,...) เราต้องแปลงผลบวกนั้นให้เป็นเลขฐาน 8 โดยใช้หลักการหารด้วยฐาน (คือ 8)

ส่วนประกอบ	วิธีคำนวณ	ความหมาย
ผลลัพธ์ในหลักปัจจุบัน (Digit)	**ผลบวก** $(\mathrm{mod}\ 8)$ (การหารเอา **เศษ**)	คือ **เศษ** ที่เหลือจากการจัดกลุ่มเป็น 8 ชุด ซึ่งก็คือตัวเลขโดด (0-7) ที่เราจะเขียนลงไปในหลักนั้น
ตัวทด (Carry)	$\lfloor \frac{\text{ผลบวก}}{8} \rfloor$ (การหารเอา **จำนวนเต็ม**)	คือ **จำนวนครั้ง** ที่เราสามารถจัดกลุ่มผลบวกให้ครบ **8** ชุด ซึ่งแต่ละชุดเท่ากับ 1 หน่วยของหลักถัดไป

รูปที่ 2-26 หลักการคำนวณหาค่าหลัก (Digit) และตัวทด (Carry) ในระบบเลขฐานแปด

ตัวอย่างเปรียบเทียบ:

- **ฐาน 10:** $9 + 5 = 14$
  - ผลลัพธ์: $14 \pmod{10} = 4$ (เศษ)
  - ตัวทด: $\lfloor \frac{14}{10} \rfloor = 1$ (1 กลุ่มของ 10)
- **ฐาน 8:** $6 + 7 = 13$
  - ผลลัพธ์: $13 \pmod{8} = 5$ (เศษที่เหลือจากการจัด 8 ชุด)
  - ตัวทด: $\lfloor \frac{13}{8} \rfloor = 1$ (1 กลุ่มของ 8)
  - ดังนั้น $6_8 + 7_8 = \mathbf{15_8}$

ตัวอย่างการบวกโดยละเอียด: $576_8 + 345_8$

หลัก	การบวกในฐาน 10	การหารด้วย 8 (Octal Conversion)	ผลลัพธ์ฐาน 8
Carry			**11**
576			576_8
+ 345			$+345_8$
Result			$\mathbf{1143_8}$
หลักที่ 0	$6 + 5 = \mathbf{11}$	$11 \div 8 = \mathbf{1}$ เศษ $\mathbf{3}$	$\mathbf{3}$ (ทด $\mathbf{1}$)
หลักที่ 1	$7 + 4 + \mathbf{1}(\text{ทด}) = \mathbf{12}$	$12 \div 8 = \mathbf{1}$ เศษ $\mathbf{4}$	$\mathbf{4}$ (ทด $\mathbf{1}$)
หลักที่ 2	$5 + 3 + \mathbf{1}(\text{ทด}) = \mathbf{9}$	$9 \div 8 = \mathbf{1}$ เศษ $\mathbf{1}$	$\mathbf{1}$ (ทด $\mathbf{1}$)
หลักที่ 3	$\mathbf{1}(\text{ทด}) + 0 + 0 = \mathbf{1}$	$1 \div 8 = \mathbf{0}$ เศษ $\mathbf{1}$	$\mathbf{1}$ (ทด $\mathbf{0}$)

รูปที่ 2-27 ตัวอย่างขั้นตอนการบวกเลขฐานแปดโดยละเอียด $576ฐาน8 + 345ฐาน8

คำอธิบายทีละขั้นตอน

1. **หลักที่ 0 (ขวาสุด):**
   - บวก: 6 + 5 = 11 (ในฐาน 10)
   - **เกิน 7** จึงต้องทด: 11 มีกลุ่ม 8 อยู่ 1 กลุ่ม (ตัวทด **1**) เหลือเศษ 3 (ผลลัพธ์)
   - เขียน **3** ใส่หลักที่ 0, ทด **1** ไปหลักที่ 1
2. **หลักที่ 1:**
   - บวก: 7 + 4 + 1 (ตัวทด) = 12 (ในฐาน 10)
   - **เกิน 7** จึงต้องทด: 12 มีกลุ่ม 8 อยู่ 1 กลุ่ม (ตัวทด **1**) เหลือเศษ 4 (ผลลัพธ์)
   - เขียน **4** ใส่หลักที่ 1, ทด **1** ไปหลักที่ 2
3. **หลักที่ 2:**
   - บวก: 5 + 3 + 1 (ตัวทด) = 9 (ในฐาน 10)
   - **เกิน 7** จึงต้องทด: 9 มีกลุ่ม 8 อยู่ 1 กลุ่ม (ตัวทด **1**) เหลือเศษ 1 (ผลลัพธ์)
   - เขียน **1** ใส่หลักที่ 2, ทด **1** ไปหลักที่ 3
4. **หลักที่ 3 (ซ้ายสุด):**
   - บวก: 1 (ตัวทด) + 0 + 0 = 1
   - 1 **ไม่เกิน 7** จึงไม่เกิดการทด
   - เขียน **1** ใส่หลักที่ 3

ดังนั้น: $576_8 + 345_8 = 1143_8$

4 การตรวจสอบความถูกต้อง (Verification in Decimal)

เพื่อความมั่นใจว่าเข้าใจถูกต้อง เราจะแปลงกลับไปเป็นฐาน 10 เพื่อเปรียบเทียบ

- **ตัวตั้ง:** $576_8 = (5 * 8^2) + (7 * 8^1) + (6\ 8^0) = 320 + 56 + 6 = 382_{10}$
- **ตัวบวก:** $345_8 = (3 * 8^2) + (4 * 8^1) + (5 * 8^0) = 192 + 32 + 5 = 229_{10}$
- **บวกในฐาน 10:** $382 + 229 = 611_{10}$
- **ผลลัพธ์ฐาน 8:** $1143_8 = (1 * 8^3) + (1 * 8^2) + (4 * 8^1) + (3 * 8^0)$
    - $1143_8 = 512 + 64 + 32 + 3 = 611_{10}$

  ผลลัพธ์ 611_{10} ตรงกัน! แสดงว่าการบวกฐาน 8 นั้นถูกต้องแล้ว

7.4 การลบเลขฐาน 8 (Octal Subtraction)

1. หลักการพื้นฐานของการ "ยืม"

การลบเลขฐาน 8 มีหลักการคล้ายกับการลบเลขฐาน 10 แต่มีข้อแตกต่างที่สำคัญคือ เมื่อตัวเลขในหลักบนมีค่าน้อยกว่าตัวล่างและต้องทำการ **"ยืม"** จากหลักถัดไป (ซ้ายมือ):

- **ในเลขฐาน 10:** เมื่อยืมจากหลักข้างหน้า จะได้มา **10** หน่วย
- **ในเลขฐาน 8:** เมื่อยืมจากหลักข้างหน้า จะได้มา **8** หน่วย

กฎสำคัญที่สุดในการลบ: เมื่อมีการยืมจากหลักข้างหน้า **1 หน่วย** ค่าในหลักปัจจุบันจะเพิ่มขึ้น **8**

2. ตัวอย่างการลบโดยละเอียด: $541_8 - 165_8$ (ตัวอย่างที่ต้องมีการยืมหลายครั้งเพื่อแสดงให้เห็นถึงหลักการ)

หลัก	การคำนวณในหลักนั้น	ผลลัพธ์ (ฐาน 10)	การยืมและการลบ (ฐาน 8)	ผลลัพธ์ฐาน 8
Borrow			**1 1 คือ การยืม**	
541			5 4 1_8	
- 165			- 1 6 5_8	
Result			3 5 4_8	
หลักที่ 0	1 – 5	-4 (ติดลบ)	ยืมจากหลัก 1 (4 เหลือ **3**) ได้ 8 มาบวก เป็น (8+1) - 5 = 4	4
หลักที่ 1	4 - 6 (ตอนนี้คือ 3-6)	-3 (ติดลบ)	ยืมจากหลัก 2 (5 เหลือ **4**), ได้ 8 มาบวกเป็น (8+3) - 6 = 5	5
หลักที่ 2	5 – 1 (ตอนนี้คือ 4-1)	3	4 (ที่เหลือจากการยืม) - 1 = 3	3

ตารางที่ 2-4 ตารางเปรียบเทียบค่าเลขฐานสิบ ฐานสอง ฐานแปด และฐานสิบหก (0-15)

ดังนั้น: $541_8 - 165_8 = 354_8$

คำอธิบายทีละขั้นตอน (เน้นการยืม 8 หน่วย)

1. **หลักที่ 0 (ขวาสุด):**
   - 1 - 5 **ลบไม่ได้** เพราะ 1 < 5
   - ต้อง **ยืม 1** จากหลักที่ 1 (หลัก "4") ทำให้เลข **4** ในหลักที่ 1 เหลือเพียง **3**
   - ค่าที่ยืมมา 1 หน่วยในหลักที่ 1 มีค่าเท่ากับ **8** หน่วยในหลักที่ 0
   - คำนวณ: (8 + 1) - 5 = 9 - 5 = 4
   - เขียน 4 ในผลลัพธ์หลักที่ 0
2. **หลักที่ 1:**
   - ตอนนี้ตัวตั้งเหลือ **3** (ถูกยืมไปแล้ว)
   - 3 - 6 **ลบไม่ได้** เพราะ 3 < 6
   - ต้อง **ยืม 1** จากหลักที่ 2 (หลัก "5") ทำให้เลข **5** ในหลักที่ 2 เหลือเพียง **4**
   - ค่าที่ยืมมา 1 หน่วยในหลักที่ 2 มีค่าเท่ากับ **8** หน่วยในหลักที่ 1
   - คำนวณ: (8 + 3) - 6 = 11 - 6 = 5
   - เขียน 5 ในผลลัพธ์หลักที่ 1
3. **หลักที่ 2 (ซ้ายสุด):**
   - ตอนนี้ตัวตั้งเหลือ **4** (ถูกยืมไปแล้ว)
   - 4 - 1 = 3
   - เขียน 3 ในผลลัพธ์หลักที่ 2

3 การตรวจสอบความถูกต้อง (Verification in Decimal) เราจะแปลงกลับไปเป็นฐาน 10 เพื่อเปรียบเทียบ

- **ตัวตั้ง:** $541_8 = (5 * 8^2) + (4 * 8^1) + (1 * 8^0) = 320 + 32 + 1 = \mathbf{353_{10}}$
- **ตัวลบ:** $165_8 = (1 * 8^2) + (6 * 8^1) + (5 * 8^0) = 64 + 48 + 5 = \mathbf{117_{10}}$
- **ลบในฐาน 10:** $353 - 117 = \mathbf{236_{10}}$
- **ผลลัพธ์ฐาน 8:** $354_8 = (3 * 8^2) + (5 * 8^1) + (4 * 8^0)$
  - $\mathbf{354_8} = (3 * 64) + (5 * 8) + (4 * 1)$
  - $\mathbf{354_8} = 192 + 40 + 4 = \mathbf{236_{10}}$

สรุป ผลลัพธ์ 236_{10} ตรงกัน แสดงว่าหลักการ "ยืม 8" ถูกต้อง

7.5 การบวกเลขฐานสิบหก (Hexadecimal Addition)

การบวกเลขฐานสิบหก (Base-16) เป็นการดำเนินการทางคณิตศาสตร์ที่คล้ายกับการบวกเลขฐานสิบ (Base-10) ที่เราคุ้นเคย แต่มีความแตกต่างที่สำคัญคือ **จำนวนหลัก** และ **ค่าของการทด (Carry)**

ทฤษฎีที่เกี่ยวข้อง เลขฐานสิบหกใช้สัญลักษณ์ 16 ตัวเพื่อแสดงค่า:

- **0 1 2 3 4 5 6 7 8 9 A B C D E F**
- 0–9 (สำหรับค่าศูนย์ถึงเก้า) **A-F** (สำหรับค่าสิบถึงสิบห้า)

ขั้นตอนการบวก :

หลักการบวกเลขฐานสิบหกมีดังนี้:

1. **บวกแต่ละหลัก:** เริ่มจากหลักขวาสุด (หลักที่มีค่าน้อยที่สุด) แล้วบวกค่าฐานสิบที่เทียบเท่ากันของสัญลักษณ์ในหลักนั้น
2. **พิจารณาผลรวม:**
    - **ถ้าผลรวม ≤ 15 :** ให้เขียนสัญลักษณ์ฐานสิบหกของผลรวมนั้นในตำแหน่งคำตอบ และไม่มีการทด (Carry) ไปยังหลักถัดไป
    - **ถ้าผลรวม ≥ 16 :** จะต้องมีการทด (Carry)
        - **ค่าที่เขียนลงในคำตอบ:** คือ **เศษ** ที่ได้จากการหารผลรวมด้วย 16 (ผลรวม (mod 16)หรือ ผลรวม - 16 * Carry) แล้วแปลงเศษนั้นกลับเป็นสัญลักษณ์ฐานสิบหก
        - **ค่าทด (Carry):** คือ **ผลหารที่เป็นจำนวนเต็ม** ที่ได้จากการหารผลรวมด้วย 16 โดยค่าทดนี้จะถูกบวกเพิ่มในหลักถัดไปทางซ้าย

➕ ตัวอย่าง: 3EFA + 25F2

การดำเนินการบวกทีละหลักจากขวาไปซ้าย โดยแปลงสัญลักษณ์เป็นค่าฐานสิบก่อนเพื่อความง่ายในการคำนวณ

1. หลักที่ 1 (หลักขวาสุด) - A + 2

- **แปลงค่า:** $A_{16} = 10_{10}$
- **ผลรวม:** $10 + 2 = 12_{10}$
- **พิจารณา:** $12 \leq 15$
- **คำตอบ:** 12_{10} เทียบเท่า C_{16}
- **ทด (Carry):** 0

	3	*E*	*F*	*A*
+	2	5	*F*	2
Carry:	0	0	0	0
Sum:				*C*

2. หลักที่ 2 - F + F + Carry (0)

- **แปลงค่า:** $F_{16} = 15_{10}$
- **ผลรวม:** $15 + 15 + 0 = 30_{10}$
- **พิจารณา:** $30 \geq 16$ (ต้องทด)
  - **ค่าทด (Carry):** 30 หาร 16
  - **ค่าในคำตอบ:** $30 - (1 * 16) = 14_{10}$
- **คำตอบ:** 14_{10} เทียบเท่า E_{16}
- **ทด (Carry) :** 1

	1			
	3	E	F	A
$+$	2	5	F	2
Carry:	0	1	0	0
Sum:			E	C

3. หลักที่ 3 - E + 5 + Carry (1)

- **แปลงค่า:** $E_{16} = 14_{10}$
- **ผลรวม:** $14 + 5 + 1 = 20_{10}$
- **พิจารณา:** $20 \geq 16$ (ต้องทด)
  - **ค่าทด (Carry):** 20 / 16 = 1
  - **ค่าในคำตอบ:** $20 - (1 * 16) = 4_{10}$
- **คำตอบ:** 4_{10} เทียบเท่า 4_{16}
- **ทด (Carry):** 1

	1			
	3	E	F	A
$+$	2	5	F	2
Carry:	1	1	0	0
Sum:		4	E	C

4. หลักที่ 4 - 3 + 2 + Carry (1)

- **ผลรวม:** $3 + 2 + 1 = 6_{10}$
- **พิจารณา:** $6 \leq 15$
- **คำตอบ:** 6_{10} เทียบเท่า 6_{16}
- **ทด (Carry):** 0

	3	E	F	A
$+$	2	5	F	2
Carry:	0	1	1	0
Sum:	6	4	E	C

สรุปผลรวม: $3EFA_{16} + 25F2_{16} = 64EC_{16}$

ตัวอย่างเพิ่มเติม

บวกเลข $A3D_{16} + 9E_{16}$

1. หลักที่ 1 (D + E)

- **แปลงค่า:** D=13, E=14
- **ผลรวม:** $13 + 14 = 27_{10}$
- **คำตอบ:** $27 \pmod{16} = 11_{10} = B_{16}$
- **ทด (Carry) :** $(27 / 16) = 1$

2. หลักที่ 2 (3 + 9 + Carry 1)

- **ผลรวม:** $3 + 9 + 1 = 13_{10}$
- **คำตอบ:** $13 \leq 15$, $13_{10} = D_{16}$
- **ทด (Carry):** 0

3. หลักที่ 3 (A + 0 + Carry 0)

- **แปลงค่า:** A=10
- **ผลรวม:** $10 + 0 + 0 = 10_{10}$
- **คำตอบ:** $10_{10} = A_{16}$

คำตอบสุดท้าย: $A3D_{16} + 9E_{16} = ADB_{16}$

7.6 การลบเลขฐานสิบหก (Hexadecimal Subtraction)

การลบเลขฐานสิบหก (Base-16) เป็นการดำเนินการที่หลักการคล้ายกับการลบเลขฐานสิบ แต่ต้องจำว่าสัญลักษณ์ A ถึง F มีค่าเท่ากับ 10 ถึง 15 ในฐานสิบ และเมื่อมีการยืมจากหลักทางซ้าย จะได้ค่าเพิ่มมา 16 เสมอ

ทฤษฎีการยืม (Borrowing Principle)

เมื่อตัวเลขในหลักที่เรากำลังลบ **มีค่าน้อยกว่า** ตัวลบ เราจะต้องทำการ **ยืม** จากหลักที่อยู่ทางซ้ายมือถัดไป:

1. ลดค่าของตัวเลขในหลักที่ถูกยืมลง 1 (เหมือนการลบในฐานสิบ)
2. เพิ่มค่าให้กับตัวเลขในหลักปัจจุบันด้วย 16 (เพราะ 1 ที่ถูกยืมมาจากหลักซ้ายมือมีค่าเท่ากับ $1 * 16^n$ ซึ่งเมื่อนำมาใช้ในหลักปัจจุบันที่มีค่าน้อยกว่า จะเท่ากับ 16 ของหน่วยในหลักนั้น)
3. ดำเนินการลบด้วยค่าใหม่ที่เพิ่ม 16 เข้าไปแล้ว

ขั้นตอนการลบและตัวอย่าง

ยกตัวอย่างการลบ **$73A5_{16}$ - $2FA_{16}$** โดยดำเนินการทีละหลักจากขวาไปซ้าย

1. หลักที่ 1 (หลักขวาสุด): 5 - A

- **แปลงค่า:** $5_{16} = 5_{10}$, $A_{16} = 10_{10}$
- **เปรียบเทียบ:** 5 มีค่าน้อยกว่า 10 **ต้องยืม** จากหลักที่ 2 (หลัก A)
- **ยืมจากหลักที่ 2:** หลัก A จะถูกลดค่าลง 1 เหลือ 9 (A_{16} - 1 = 9_{16})
- **ค่าในหลักที่ 1:** จะเพิ่มขึ้น 16 กลายเป็น 5 + 16 = 21_{10}
- **ผลลัพธ์:** 21_{10} - 10_{10} = 11_{10}
- **คำตอบ:** 11_{10} เทียบเท่า B_{16}

Borrow:			9	
	7	3	A	5
−		2	F	A
Difference:				B

2. หลักที่ 2 (ที่ถูกยืม) - 9 - F

- **แปลงค่า:** ค่าใหม่ในหลักนี้คือ $9_{16} = 9_{10}$, $F_{16} = 15_{10}$
- **เปรียบเทียบ:** 9 มีค่าน้อยกว่า 15 **ต้องยืม** จากหลักที่ 3 (หลัก 3)
- **ยืมจากหลักที่ 3:** หลัก 3 จะถูกลดค่าลง 1 เหลือ 2 (3_{16} - 1 = 2_{16})
- **ค่าในหลักที่ 2:** จะเพิ่มขึ้น 16 กลายเป็น 9 + 16 = 25_{10}
- **ผลลัพธ์:** 25_{10} - 15_{10} = 10_{10}
- **คำตอบ:** 10_{10} เทียบเท่า A_{16}

Borrow:		2	9	
	7	3	A	5
−		2	F	A
Difference:			A	B

3. หลักที่ 3 (ที่ถูกยืม) - 2 - 2

- **แปลงค่า:** ค่าใหม่ในหลักนี้คือ $2_{16} = 2_{10}$, $2_{16} = 2_{10}$
- **ผลลัพธ์:** 2 - 2 = 0_{10}
- **คำตอบ:** 0_{16}

Borrow:		2	9	
	7	3	A	5
−		2	F	A
Difference:			A	B

4. หลักที่ 4 - 7 - 0

- **ผลลัพธ์:** 7 - 0 = 7_{10}
- **คำตอบ:** 7_{16}

Borrow:		2	9	
	7	3	A	5
−		2	F	A
Difference:	7	0	A	B

สรุปคำตอบ $73A5_{16}$ - $2FA_{16}$ = $70AB_{16}$

ตัวอย่างเพิ่มเติม: การลบที่มีการยืมต่อเนื่อง

ลองลบเลข **900_{16} - 1C_{16}**

1. หลักที่ 1 (0 - C)

- **แปลงค่า:** $0_{16} = 0_{10}$, $C_{16} = 12_{10}$
- $0 < 12$ **ต้องยืม** จากหลักที่ 2 (หลัก 0) แต่หลักที่ 2 เป็น 0 จึงต้องไปยืมต่อจากหลักที่ 3

2. การยืมต่อเนื่อง

- **ยืมจากหลักที่ 3 (9):** 9 ถูกลดเหลือ 8
- **หลักที่ 2 (0):** จะได้ 0+16 = 16. จากนั้นหลักที่ 2 จะถูกยืมไป 1 สำหรับหลักที่ 1 ทำให้เหลือ 16 - 1 = 15 (F_{16})
- **หลักที่ 1 (0):** จะได้ 0+16 = 16

Borrow:	8	*F*	
	9	0	0
–		1	*C*
Difference:			

3. ดำเนินการลบ

- **หลักที่ 1:** $16_{10} - 12_{10} = 4_{10}$ เท่ากับ 4_{16}
- **หลักที่ 2 (ค่าใหม่ F=15):** $15_{10} - 1_{10} = 14_{10}$ เท่ากับ E_{16}
- **หลักที่ 3 (ค่าใหม่ 8):** $8 - 0 = 8_{10}$ เท่ากับ 8_{16}

คำตอบสุดท้าย: $900_{16} - 1C_{16} = 8E4_{16}$

7.7 การลบเลขด้วย 1's Complement (One's Complement Subtraction)

การลบเลขฐานสองด้วยวิธี 1's Complement เป็นวิธีหนึ่งที่คอมพิวเตอร์ใช้ในการดำเนินการลบ โดยเปลี่ยนการลบให้เป็นการบวก ซึ่งช่วยลดความซับซ้อนในการออกแบบวงจร

หลักการของ 1's Complement:

1. **หา 1's Complement ของตัวลบ:** เปลี่ยนบิต 0 ให้เป็น 1 และเปลี่ยนบิต 1 ให้เป็น 0 ในทุกๆ บิตของตัวลบ
2. **นำตัวตั้งมาบวกกับ 1's Complement ของตัวลบ:** ทำการบวกเลขฐานสองตามปกติ
3. **พิจารณาตัวทด (End-Around Carry):**
    - ถ้ามีตัวทด (Carry) เกิดขึ้นที่บิตซ้ายสุด (บิตที่มีนัยสำคัญสูงสุด - MSB) ให้นำตัวทดนั้นไป **บวกเพิ่ม** เข้ากับบิตขวาสุด (บิตที่มีนัยสำคัญน้อยที่สุด - LSB) ของผลลัพธ์
    - ถ้าไม่มีตัวทดเกิดขึ้นที่บิตซ้ายสุด แสดงว่าผลลัพธ์เป็นค่าลบ และผลลัพธ์ที่ได้ (ไม่รวมบิตทด) จะต้องหา 1's Complement อีกครั้ง แล้วใส่เครื่องหมายลบนำหน้า

ตัวอย่างที่ 1: ตัวตั้ง > ตัวลบ (ผลลัพธ์เป็นบวก)

โจทย์: ลบ $1011_2 - 0110_2$ (เท่ากับ 11−6 ในฐาน 10)

วิธีทำ:

1. **ตัวตั้ง:** 1011_2
2. **ตัวลบ:** 0110_2
3. **หา 1's Complement ของตัวลบ:**

   0110_2

   rightarrow1001_2
4. **นำตัวตั้งมาบวกกับ 1's Complement ของตัวลบ:**

   1011_2

   $+ 1001_2$

   $1\ 0100_2$ <-- มีตัวทด (Carry) เกิดขึ้นที่บิตซ้ายสุด (1)
5. **นำตัวทดไปบวกเพิ่มที่บิตขวาสุด (End-Around Carry):**

   0100_2

   $+ \quad 1$ <-- ตัวทด

   0101_2

   ผลลัพธ์คือ 0101_2 (เท่ากับ 5 ในฐาน 10)

ตัวอย่างที่ 2: ตัวตั้ง < ตัวลบ (ผลลัพธ์เป็นลบ)

โจทย์: ลบ $0110_2 - 1011_2$ (เท่ากับ 6−11 ในฐาน 10)

วิธีทำ:

1. **ตัวตั้ง:** 0110_2
2. **ตัวลบ:** 1011_2
3. **หา 1's Complement ของตัวลบ:**

   1011_2

   rightarrow0100_2
4. **นำตัวตั้งมาบวกกับ 1's Complement ของตัวลบ:**

0110_2

$+ 0100_2$

1010_2 <-- ไม่มีตัวทดเกิดขึ้นที่บิตซ้ายสุด

5. **ไม่มีตัวทด:** แสดงว่าผลลัพธ์เป็นลบ และต้องหา 1's Complement ของผลลัพธ์ที่ได้

   1010_2

   rightarrow0101_2

6. **ใส่เครื่องหมายลบนำหน้า:** -0101_2

ผลลัพธ์คือ -0101_2 (เท่ากับ -5 ในฐาน 10)

7.8 การลบเลขด้วย 2's Complement (Two's Complement Subtraction)

การลบเลขด้วย 2's Complement เป็นวิธีที่นิยมใช้ในคอมพิวเตอร์มากที่สุด เพราะจัดการได้ง่ายกว่า 1's Complement (ไม่ต้องมี End-Around Carry)

หลักการของ 2's Complement:

1. **หา 2's Complement ของตัวลบ:**
   - **วิธีที่ 1 (ง่ายที่สุด):** หา 1's Complement ของตัวลบก่อน แล้วบวก 1 เข้าไปที่บิตขวาสุด
   - **วิธีที่ 2:** เริ่มจากบิตขวาสุดของตัวลบ หาบิต 1 ตัวแรก (นับจากขวาไปซ้าย) เก็บค่าบิตทั้งหมดจากขวาไปจนถึงบิต 1 ตัวแรกไว้เหมือนเดิม จากนั้นสลับบิตที่เหลือทั้งหมด (0 เป็น 1, 1 เป็น 0)
2. **นำตัวตั้งมาบวกกับ 2's Complement ของตัวลบ:** ทำการบวกเลขฐานสองตามปกติ
3. **พิจารณาตัวทด (Carry):**
   - **ถ้ามีตัวทดเกิดขึ้นที่บิตซ้ายสุด:** ให้ **ทิ้ง** ตัวทดนั้นไป ผลลัพธ์ที่ได้คือคำตอบที่เป็นบวก
   - **ถ้าไม่มีตัวทดเกิดขึ้นที่บิตซ้ายสุด:** แสดงว่าผลลัพธ์เป็นค่าลบ และผลลัพธ์ที่ได้ (ไม่รวมบิตทด) จะต้องหา 2's Complement อีกครั้ง แล้วใส่เครื่องหมายลบนำหน้า

ตัวอย่างที่ 1: ตัวตั้ง > ตัวลบ (ผลลัพธ์เป็นบวก)

โจทย์: ลบ $1011_2 - 0110_2$ (เท่ากับ 11−6 ในฐาน 10)

วิธีทำ:

1. **ตัวตั้ง:** 1011_2
2. **ตัวลบ:** 0110_2
3. **หา 2's Complement ของตัวลบ:**

- 1's Complement ของ 0110_2 คือ 1001_2
- บวก 1 เข้าไป: $1001_2+1_2=1010_2$ (นี่คือ 2's Complement ของ 0110_2)

4. **นำตัวตั้งมาบวกกับ 2's Complement ของตัวลบ:**

   1011_2

   $+ 1010_2$

   $1\ 0101_2$ <-- มีตัวทด (Carry) เกิดขึ้นที่บิตซ้ายสุด (1)

5. **ทิ้งตัวทด:** ทิ้งบิต 1 ที่เป็นตัวทดไป

ผลลัพธ์คือ 0101_2 (เท่ากับ 5 ในฐาน 10)

ตัวอย่างที่ 2: ตัวตั้ง < ตัวลบ (ผลลัพธ์เป็นลบ)

โจทย์: ลบ 0110_2-1011_2 (เท่ากับ 6−11 ในฐาน 10)

วิธีทำ:

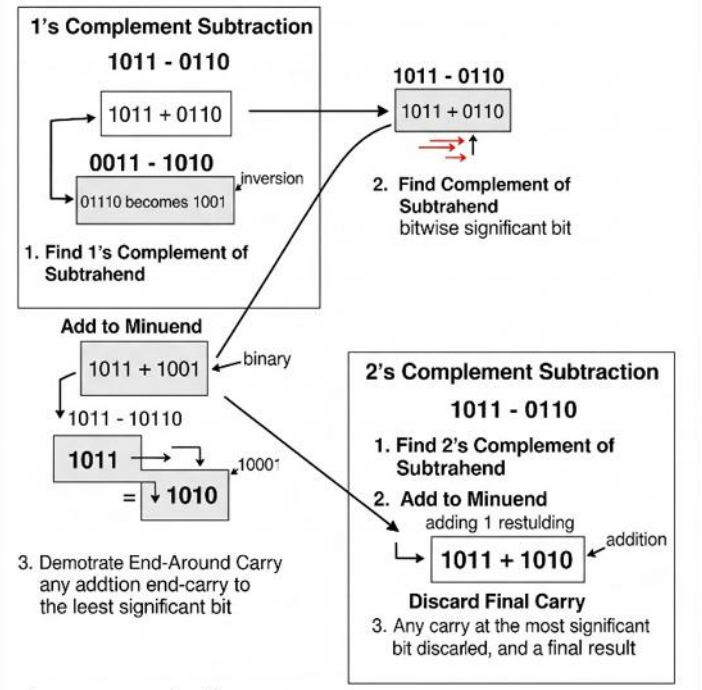
1. **ตัวตั้ง:** 0110_2
2. **ตัวลบ:** 1011_2
3. **หา 2's Complement ของตัวลบ:**
   - 1's Complement ของ 1011_2 คือ 0100_2
   - บวก 1 เข้าไป: $0100_2+1_2=0101_2$ (นี่คือ 2's Complement ของ 1011_2)
4. **นำตัวตั้งมาบวกกับ 2's Complement ของตัวลบ:**

   0110_2

   + 0101_2

   1011_2 <-- ไม่มีตัวทดเกิดขึ้นที่บิตซ้ายสุด

5. **ไม่มีตัวทด:** แสดงว่าผลลัพธ์เป็นลบ และต้องหา 2's Complement ของผลลัพธ์ที่ได้
   - 1's Complement ของ 1011_2 คือ 0100_2
   - บวก 1 เข้าไป: $0100_2+1_2=0101_2$
6. **ใส่เครื่องหมายลบนำหน้า:** -0101_2
7. **ผลลัพธ์คือ -0101_2 (เท่ากับ -5 ในฐาน 10)**



รูปที่ 2-28 ตัวอย่างขั้นตอนการบวกเลขฐานสิบหกโดยคอมพิวเตอร์

จากภาพแสดงวิธีการลบเลขฐานสองโดยใช้ Complement ซึ่งเป็นวิธีที่คอมพิวเตอร์ใช้ในการดำเนินการลบโดยเปลี่ยนให้เป็นการบวก เพื่อความง่ายในการออกแบบวงจร:

- **1's Complement Subtraction (การลบด้วย 1's Complement):**
  - **ขั้นตอนที่ 1: Find 1's Complement of Subtrahend (หา 1's Complement ของตัวลบ):** เปลี่ยนบิต 0 เป็น 1 และบิต 1 เป็น 0 ในทุกๆ บิตของตัวลบ
  - **ขั้นตอนที่ 2: Add to Minuend (บวกกับตัวตั้ง):** นำตัวตั้งมาบวกกับ 1's Complement ของตัวลบ
  - **ขั้นตอนที่ 3: Handle End-Around Carry (จัดการตัวทดวนรอบ):**
    - ถ้ามีตัวทด (Carry) ที่บิตซ้ายสุด (บิตที่มีนัยสำคัญสูงสุด) ให้นำตัวทดนั้นไป **บวกเพิ่ม** เข้ากับบิตขวาสุด (บิตที่มีนัยสำคัญน้อยที่สุด) ของผลลัพธ์
    - ถ้าไม่มีตัวทด แสดงว่าผลลัพธ์เป็นค่าลบ และต้องหา 1's Complement ของผลลัพธ์ที่ได้ แล้วใส่เครื่องหมายลบนำหน้า
- **2's Complement Subtraction (การลบด้วย 2's Complement):**
  - **ขั้นตอนที่ 1: Find 2's Complement of Subtrahend (หา 2's Complement ของตัวลบ):** ทำได้โดยหา 1's Complement ของตัวลบก่อน แล้วบวก 1 เข้าไปที่บิตขวาสุด
  - **ขั้นตอนที่ 2: Add to Minuend (บวกกับตัวตั้ง):** นำตัวตั้งมาบวกกับ 2's Complement ของตัวลบ
  - **ขั้นตอนที่ 3: Discard Final Carry (ทิ้งตัวทดสุดท้าย):**
    - ถ้ามีตัวทด (Carry) ที่บิตซ้ายสุด ให้ **ทิ้ง** ตัวทดนั้นไป ผลลัพธ์ที่ได้คือคำตอบที่เป็นบวก
    - ถ้าไม่มีตัวทด แสดงว่าผลลัพธ์เป็นค่าลบ และต้องหา 2's Complement ของผลลัพธ์ที่ได้ แล้วใส่เครื่องหมายลบนำหน้า

8. การแทนค่าข้อมูลด้วยรหัส (Data Encoding)

การแทนค่าข้อมูลด้วยรหัส คือกระบวนการแปลงข้อมูลจากรูปแบบที่มนุษย์อ่านเข้าใจ (เช่น ตัวอักษร, ตัวเลข, สัญลักษณ์) ไปเป็นรูปแบบที่คอมพิวเตอร์สามารถจัดเก็บและประมวลผลได้ (คือชุดของบิต 0 และ 1) รหัสแต่ละตัวจะถูกกำหนดให้แทนค่าเฉพาะ เพื่อให้คอมพิวเตอร์สามารถตีความข้อมูลได้อย่างถูกต้อง

8.1 รหัส ASCII (American Standard Code for Information Interchange)

รหัส ASCII เป็นหนึ่งในมาตรฐานการเข้ารหัสตัวอักษรที่เก่าแก่และเป็นที่นิยมที่สุด โดยมีวัตถุประสงค์เพื่อใช้แทนตัวอักษรภาษาอังกฤษ ตัวเลข และสัญลักษณ์ต่างๆ ในคอมพิวเตอร์

- **Standard ASCII (ASCII มาตรฐาน):**
    - มีขนาด **7 บิต**
    - สามารถแทนตัวอักษรได้ทั้งหมด 2^7=128 ตัว (ตั้งแต่รหัส 000 ถึง 127)
    - ครอบคลุมตัวอักษรภาษาอังกฤษตัวพิมพ์ใหญ่ (A-Z), ตัวพิมพ์เล็ก (a-z), ตัวเลข (0-9), สัญลักษณ์ทางคณิตศาสตร์ (เช่น +, -, *, /), เครื่องหมายวรรคตอน (เช่น . , ; :), และรหัสควบคุม (Control Codes) ที่ใช้ในการสื่อสารหรือควบคุมอุปกรณ์ (เช่น Enter, Tab, Backspace)
- **Extended ASCII (ASCII เพิ่มเติม/ขยาย):**
    - มีขนาด **8 บิต**
    - สามารถแทนตัวอักษรได้ทั้งหมด 2^8=256 ตัว (ตั้งแต่รหัส 000 ถึง 255)
    - เพิ่มเติมจาก Standard ASCII โดยใช้บิตที่ 8 (บิตซ้ายสุด) ในการเพิ่มตัวอักษรพิเศษต่างๆ เช่น ตัวอักษรที่มีเครื่องหมายกำกับ (Accent Marks), สัญลักษณ์กราฟิก, สัญลักษณ์สกุลเงิน หรือตัวอักษรของภาษาอื่นๆ เช่น ภาษายุโรปบางภาษา
    - Extended ASCII มีหลายเวอร์ชัน ขึ้นอยู่กับผู้ผลิตหรือแพลตฟอร์ม ทำให้เกิดปัญหาความไม่เข้ากันได้ของรหัสในบางครั้ง (เช่น CP437, Latin-1)

ทำไม ASCII ถึงสำคัญ?

ASCII เป็นพื้นฐานที่ทำให้คอมพิวเตอร์จากผู้ผลิตและระบบปฏิบัติการที่แตกต่างกันสามารถสื่อสารและแลกเปลี่ยนข้อมูลตัวอักษรพื้นฐานกันได้ ตัวอย่างเช่น เมื่อคุณพิมพ์ตัวอักษร 'A' บนคีย์บอร์ด คอมพิวเตอร์จะแปลง 'A' ให้เป็นรหัส ASCII ที่เป็นเลขฐานสอง (01000001) และจัดเก็บไว้ จากนั้นเมื่อคอมพิวเตอร์ต้องการแสดงผล 'A' บนหน้าจอ ก็จะแปลงรหัส 01000001 กลับเป็นตัวอักษร 'A' ที่เรามองเห็น

8.2 ตารางการเปรียบเทียบรหัส ASCII

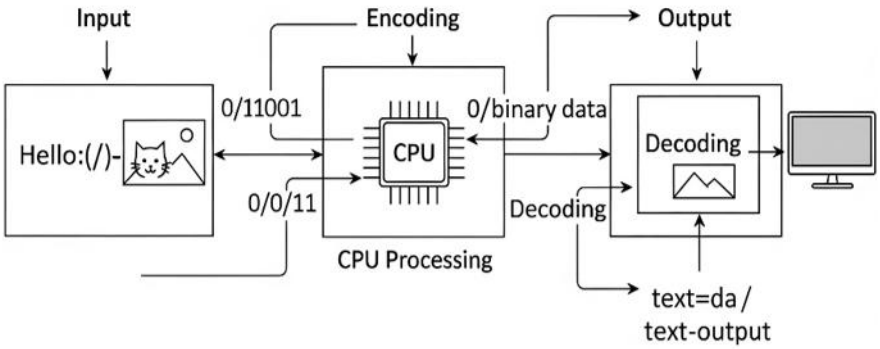
ตาราง นี้จะแสดงรหัส ASCII บางส่วน เพื่อให้เห็นความสัมพันธ์ระหว่างตัวอักษร, เลขฐานสิบ, เลขฐานสอง, และเลขฐานสิบหก

ตัวอักษร/สัญลักษณ์	รหัส Decimal (ฐาน 10)	รหัส Binary (ฐาน 2) จำนวน (7 บิต)	รหัส Hexadecimal (ฐาน 16)
Control Characters (รหัสควบคุม)			
NUL (Null)	0	0000000	00
SOH (Start of Heading)	1	0000001	01
STX (Start of Text)	2	0000010	02
CR (Carriage Return)	13	0001101	0D
LF (Line Feed)	10	0001010	0A
ESC (Escape)	27	0011011	1B
DEL (Delete)	127	1111111	7F
Printable Characters (ตัวอักษรที่พิมพ์ได้)			
(Space)	32	0100000	20
!	33	0100001	21
"	34	0100010	22
#	35	0100011	23
0	48	0110000	30
1	49	0110001	31
2	50	0110010	32
9	57	0111001	39
:	58	0111010	3A
;	59	0111011	3B
<	60	0111100	3C

ตัวอักษร/สัญลักษณ์	รหัส Decimal (ฐาน 10)	รหัส Binary (ฐาน 2) จำนวน (7 บิต)	รหัส Hexadecimal (ฐาน 16)
=	61	0111101	3D
>	62	0111110	3E
?	63	0111111	3F
@	64	1000000	40
A	65	1000001	41
B	66	1000010	42
C	67	1000011	43
Z	90	1011010	5A
[	91	1011011	5B
\	92	1011100	5C
]	93	1011101	5D
^	94	1011110	5E
_	95	1011111	5F
`	96	1100000	60
a	97	1100001	61
b	98	1100010	62
c	99	1100011	63
z	122	1111010	7A
{	123	1111011	7B
		124	1111100
}	125	1111101	7D
~	126	1111110	7E

ตารางที่ 2-5 ตัวอย่างรหัสมาตรฐาน ASCII และการแทนค่าในระบบเลขฐานต่าง ๆ

หมายเหตุ: ตารางข้างต้นเป็น Standard ASCII (7 บิต) ส่วน Extended ASCII จะใช้รหัสตั้งแต่ 128 ถึง 255 ซึ่งจะแตกต่างกันไปตามชุดอักขระที่ใช้



รูปที่ 2-29 แผนผังวงจรการเข้ารหัสและถอดรหัสข้อมูลในการประมวลผลของคอมพิวเตอร์

คำอธิบายภาพและรายละเอียดของรหัส ASCII:

จากภาพแสดงให้เห็นถึงวัฏจักรของการเข้ารหัสและถอดรหัสข้อมูลในระบบคอมพิวเตอร์ โดยมีรหัส ASCII เป็นตัวอย่างหลัก ซึ่งเป็นมาตรฐานที่สำคัญในการแปลงข้อมูลที่มนุษย์เข้าใจให้เป็นภาษาของคอมพิวเตอร์

1. **Input (ข้อมูลนำเข้า):**
   - เริ่มต้นจากการที่ผู้ใช้งานป้อนข้อมูลเข้าสู่คอมพิวเตอร์ เช่น การพิมพ์ตัวอักษร 'A' บนแป้นพิมพ์
   - ข้อมูลในขั้นตอนนี้ยังอยู่ในรูปแบบที่มนุษย์อ่านและเข้าใจได้
2. **Encoding (การเข้ารหัส):**
   - เมื่อตัวอักษร 'A' ถูกป้อนเข้าสู่ระบบ คอมพิวเตอร์จะใช้มาตรฐานการเข้ารหัส เช่น **รหัส ASCII** เพื่อแปลงตัวอักษรนี้ให้เป็นชุดของเลขฐานสอง (Binary) ที่ประกอบด้วย 0 และ 1 เท่านั้น
   - สำหรับตัวอักษร 'A' รหัส ASCII มาตรฐาน (7 บิต) คือ **01000001**
   - กระบวนการนี้ทำให้ข้อมูลพร้อมสำหรับการจัดเก็บและประมวลผลภายใน CPU
3. **Storage/Processing (การจัดเก็บ/ประมวลผล):**
   - ชุดของเลขฐานสอง (01000001) ที่ได้จากการเข้ารหัสจะถูกจัดเก็บในหน่วยความจำของคอมพิวเตอร์ และถูกประมวลผลโดยหน่วยประมวลผลกลาง (CPU) ตามคำสั่งต่างๆ
   - คอมพิวเตอร์ทำงานกับข้อมูลในรูปแบบ 0 และ 1 ตลอดกระบวนการนี้
4. **Decoding (การถอดรหัส):**
   - เมื่อคอมพิวเตอร์ประมวลผลเสร็จสิ้นและต้องการแสดงผลลัพธ์ออกมา (เช่น บนหน้าจอหรือเครื่องพิมพ์) ชุดของเลขฐานสองจะถูกส่งไปยังส่วนที่ทำหน้าที่ถอดรหัส

- กระบวนการถอดรหัสจะแปลงชุดเลขฐานสอง (01000001) กลับไปเป็นตัวอักษรที่มนุษย์สามารถอ่านเข้าใจได้ (เช่น ตัวอักษร 'A')

5. **Output (ข้อมูลส่งออก):**
   - ตัวอักษร 'A' ที่ถูกถอดรหัสแล้วจะถูกแสดงผลออกมาให้ผู้ใช้งานเห็น เช่น ปรากฏบนหน้าจอหรือถูกพิมพ์ออกมาทางเครื่องพิมพ์

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 2: เรื่องข้อมูลและระบบตัวเลข

ส่วนที่ 1: ข้อสอบปรนัย (Multiple Choice) จำนวน 20 ข้อ (เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว)

1. ข้อใดคือความหมายของ "ข้อมูล" ที่คอมพิวเตอร์สามารถประมวลผลได้ถูกต้องที่สุด?
   ก. ข้อมูลทุกประเภทที่มนุษย์สามารถรับรู้ได้
   ข. ข้อมูลที่อยู่ในรูปแบบสัญญาณไฟฟ้าดิจิทัล ซึ่งถูกแทนด้วยลอจิก "0" และ "1"
   ค. ข้อมูลที่ถูกบันทึกไว้ในกระดาษหรือสื่อบันทึกภายนอกเท่านั้น
   ง. ข้อมูลที่ถูกส่งผ่านเครือข่ายอินเทอร์เน็ต

   เฉลย: ข
2. เหตุใดข้อมูลที่ป้อนเข้าสู่คอมพิวเตอร์จึงต้องถูกแปลงให้อยู่ในรูปแบบ "0" และ "1"?
   ก. เพื่อประหยัดพื้นที่ในการจัดเก็บข้อมูล
   ข. เพื่อให้มนุษย์สามารถอ่านข้อมูลได้ง่ายขึ้น
   ค. เนื่องจากคอมพิวเตอร์ทำงานด้วยระบบดิจิทัลที่เข้าใจเพียงสถานะเปิด-ปิด (On-Off) เท่านั้น
   ง. เพื่อป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาต

   เฉลย: ค
3. กระบวนการที่นำข้อมูลทั่วไปมาเปลี่ยนให้อยู่ในรูปแบบที่คอมพิวเตอร์ประมวลผลได้เรียกว่าอะไร?
   ก. Decoding ข. Processing
   ค. Encoding ง. Storing

   เฉลย: ค
4. หน้าที่หลักของกระบวนการ Encoding คืออะไร?
   ก. การแปลงข้อมูลจากรูปแบบดิจิทัลกลับไปเป็นรูปแบบที่มนุษย์เข้าใจ
   ข. การเปลี่ยนข้อมูลดิบให้เป็นข้อมูลที่มีความหมาย
   ค. การแปลงข้อมูลจากรูปแบบที่มนุษย์เข้าใจไปเป็นรูปแบบเลขฐานสอง
   ง. การจัดเรียงข้อมูลให้เป็นระเบียบในหน่วยความจำ

   เฉลย: ค
5. เมื่อคอมพิวเตอร์แสดงผลลัพธ์บนหน้าจอ กระบวนการใดที่เกิดขึ้นเพื่อแปลงข้อมูลเลขฐานสองกลับเป็นตัวอักษรหรือรูปภาพที่เรามองเห็น?
   ก. Encoding ข. Storing
   ค. Decoding ง. Processing

   เฉลย: ค

6. ถ้าคอมพิวเตอร์รับข้อมูล "1100001" จากคีย์บอร์ด กระบวนการ Encoding จะต้องแปลงข้อมูลนี้ไปเป็นอะไรก่อนที่จะจัดเก็บ?

   ก. ตัวอักษร 'a'

   ข. รหัสฐานสิบ 97

   ค. ตัวเลขฐานสอง '1100001' (ซึ่งเป็นตัวอักษร 'a' ใน ASCII)

   ง. ข้อมูลนี้ถูกเข้ารหัสมาแล้ว ไม่ต้องแปลงอีก

   เฉลย: ค (การเข้ารหัสคือการเปลี่ยนจาก A ไปเป็น 01000001 ถ้ามาจากคีย์บอร์ด ข้อมูลนั้นเป็นตัวอักษรแล้วถูกแปลงเป็นไบนารี)

7. ระบบตัวเลขที่คอมพิวเตอร์ใช้ในการประมวลผลและจัดเก็บข้อมูลโดยตรงคือระบบใด?

   ก. เลขฐาน 8 ข. เลขฐาน 10

   ค. เลขฐาน 2 ง. เลขฐาน 16

   เฉลย: ค

8. เลขฐาน 16 (Hexadecimal) มีตัวเลขและสัญลักษณ์ที่ใช้แทนค่าได้กี่ตัว?

   ก. 8 ตัว (0-7) ข. 10 ตัว (0-9)

   ค. 16 ตัว (0-9 และ A-F) ง. 10 ตัว (0-9 และ A-Z)

   เฉลย: ค

9. เลขฐานแปด (Octal) ใช้ตัวเลขใดบ้างในการแสดงค่า?

   ก. 0, 1 ข. 0-7

   ค. 0-9 ง. 0-9, A-F

   เฉลย: ข

10. ค่า 1011_2 มีค่าเท่ากับเท่าใดในระบบเลขฐาน 10?

    ก. 9 ข. 10 ค. 11 ง. 12

    เฉลย: ค

11. ค่า $2C_{16}$ มีค่าเท่ากับเท่าใดในระบบเลขฐาน 10?

    ก. 40 ข. 42 ค. 44 ง. 46

    เฉลย: ค ($2\times161+12\times160=32+12=44$)

12. ค่า 15_{10} มีค่าเท่ากับเท่าใดในระบบเลขฐาน 2?

    ก. 1011_2 ข. 1101_2

    ค. 1110_2 ง. 1111_2

    เฉลย: ง

13. การแปลงเลขฐาน 2 เป็นเลขฐาน 8 สามารถทำได้โดยการแบ่งกลุ่มบิตละกี่บิตจากขวาไปซ้าย?

    ก. 2 บิต ข. 3 บิต

    ค. 4 บิต ง. 8 บิต

    เฉลย: ข

14. หน่วยที่เล็กที่สุดในการวัดขนาดข้อมูลในคอมพิวเตอร์คืออะไร?

    ก. ไบต์ (Byte) ข. บิต (Bit)

    ค. กิโลไบต์ (KB) ง. เมกะไบต์ (MB)

    เฉลย: ข

15. ข้อมูล 1 ไบต์ (Byte) ประกอบด้วยข้อมูลกี่บิต (Bit)?

    ก. 2 บิต ข. 4 บิต

    ค. 8 บิต ง. 16 บิต

    เฉลย: ค

16. การลบเลขฐานสองด้วย 1's Complement หากผลลัพธ์สุดท้ายมีตัวทด (End-around carry) เกิดขึ้นที่บิตซ้ายสุด ควรทำอย่างไรกับตัวทดนั้น?

    ก. ทิ้งตัวทดนั้นไป ข. นำตัวทดไปบวกเพิ่มที่บิตขวาสุดของผลลัพธ์

    ค. นำตัวทดไปบวกเพิ่มที่บิตซ้ายสุดของผลลัพธ์ ง. ใส่เครื่องหมายลบนำหน้าผลลัพธ์

    เฉลย: ข

17. ข้อใดคือขั้นตอนแรกในการหา 2's Complement ของเลขฐานสอง?

    ก. กลับบิตทั้งหมด (0 เป็น 1, 1 เป็น 0)

    ข. บวก 1 เข้าไปที่บิตขวาสุด

    ค. กลับบิตทั้งหมด แล้วบวก 1 เข้าไปที่บิตขวาสุด

    ง. ทิ้งบิตซ้ายสุดออกไป

    เฉลย: ค (การหา 2's complement คือ 1's complement + 1)

18. หากทำการลบเลขฐานสองด้วย 2's Complement แล้วไม่มีตัวทดเกิดขึ้นที่บิตซ้ายสุด แสดงว่าผลลัพธ์ของการลบนั้นเป็นอย่างไร?

    ก. เป็นค่าบวกที่ถูกต้องแล้ว

    ข. เป็นค่าลบ และต้องนำผลลัพธ์ที่ได้ไปหา 2's Complement อีกครั้ง

    ค. เป็นค่าลบ และต้องนำผลลัพธ์ที่ได้ไปหา 1's Complement อีกครั้ง

    ง. เป็นค่าที่ผิดพลาด

    เฉลย: ข

19. รหัส ASCII มาตรฐาน (Standard ASCII) สามารถแทนตัวอักษรได้ทั้งหมดกี่ตัวอักษร?

    ก. 64 ตัว ข. 128 ตัว

    ค. 256 ตัว ง. 512 ตัว

    เฉลย: ข

20. รหัส ASCII ถูกนำมาใช้เพื่อวัตถุประสงค์ใดเป็นหลัก?

    ก. การเข้ารหัสข้อมูลภาพและเสียง

    ข. การแทนค่าตัวอักษร ตัวเลข และสัญลักษณ์เพื่อให้คอมพิวเตอร์ประมวลผลได้

    ค. การเข้ารหัสข้อมูลเพื่อความปลอดภัยในการส่งผ่านเครือข่าย

    ง. การบีบอัดข้อมูลให้มีขนาดเล็กลง

    เฉลย: ข

ส่วนที่ 2: ข้อสอบอัตนัย (Descriptive Questions) จำนวน 7 ข้อ (โปรดอธิบายอย่างละเอียด)

1. **คำถาม:** จงอธิบายความหมายของ "ข้อมูล" ที่คอมพิวเตอร์สามารถประมวลผลได้ พร้อมยกตัวอย่างประกอบ

   แนวทางการเฉลย: ข้อมูลที่คอมพิวเตอร์ประมวลผลได้ หมายถึง ข้อมูลที่ถูกแปลงให้อยู่ในรูปแบบของสัญญาณไฟฟ้าดิจิทัล ซึ่งแทนด้วยลอจิก "0" และ "1" เท่านั้น เนื่องจากคอมพิวเตอร์เป็นอุปกรณ์อิเล็กทรอนิกส์ที่ทำงานด้วยสถานะเปิด-ปิด (On-Off) หรือมี/ไม่มีสัญญาณไฟฟ้า ตัวอย่างเช่น ตัวอักษร 'A' เมื่อป้อนเข้าคอมพิวเตอร์จะถูกแปลงเป็นรหัสฐานสอง 01000001 หรือตัวเลข 5 จะถูกแปลงเป็น 0101 เป็นต้น

2. **คำถาม:** จงอธิบายความแตกต่างและบทบาทของกระบวนการ Encoding และ Decoding ในระบบคอมพิวเตอร์

   แนวทางการเฉลย:

   - **Encoding (การเข้ารหัส):** คือกระบวนการแปลงข้อมูลจากรูปแบบที่มนุษย์เข้าใจ (เช่น ตัวอักษร, ตัวเลข, รูปภาพ, เสียง) ให้เป็นชุดของสัญญาณดิจิทัล (เลขฐานสอง 0 และ 1) เพื่อให้คอมพิวเตอร์สามารถจัดเก็บและประมวลผลได้ บทบาทคือทำให้คอมพิวเตอร์สามารถ "เข้าใจ" และ "จัดการ" กับข้อมูลที่เข้ามาจากโลกภายนอกได้
   - **Decoding (การถอดรหัส):** คือกระบวนการย้อนกลับของการ Encoding โดยแปลงชุดของสัญญาณดิจิทัล (เลขฐานสอง 0 และ 1) ที่คอมพิวเตอร์ประมวลผลแล้ว กลับไปเป็นรูปแบบที่มนุษย์สามารถรับรู้และเข้าใจได้อีกครั้ง (เช่น แสดงตัวอักษรบนหน้าจอ, เล่นเสียงเพลง, แสดงรูปภาพ) บทบาทคือทำให้คอมพิวเตอร์สามารถ "สื่อสาร" ผลลัพธ์กลับไปยังผู้ใช้งานได้

3. **คำถาม:** จงอธิบายระบบตัวเลขทั้ง 4 ชนิดที่ใช้ในคอมพิวเตอร์ (เลขฐาน 2, 8, 10, 16) โดยระบุว่าแต่ละระบบใช้ตัวเลขและสัญลักษณ์ใดบ้าง พร้อมเหตุผลที่ใช้ในคอมพิวเตอร์

   แนวทางการเฉลย:

   - **เลขฐาน 2 (Binary):** ใช้ 0 และ 1 เป็นสัญลักษณ์ เหตุผล: เป็นภาษาธรรมชาติของคอมพิวเตอร์ เพราะแทนสถานะเปิด-ปิดของวงจรไฟฟ้า
   - **เลขฐาน 8 (Octal):** ใช้ 0-7 เป็นสัญลักษณ์ เหตุผล: ใช้ในการแสดงค่าเลขฐาน 2 ที่ยาวๆ ให้สั้นลง เนื่องจาก 1 หลักของเลขฐาน 8 แทนได้ 3 บิตของเลขฐาน 2 (เช่น 78=1112)

- **เลขฐาน 10 (Decimal):** ใช้ 0-9 เป็นสัญลักษณ์ เหตุผล: เป็นระบบตัวเลขที่มนุษย์ใช้ในชีวิตประจำวัน จึงใช้ในการสื่อสารระหว่างมนุษย์กับคอมพิวเตอร์
- **เลขฐาน 16 (Hexadecimal):** ใช้ 0-9 และ A-F เป็นสัญลักษณ์ (A=10, B=11, ..., F=15) เหตุผล: ใช้ในการแสดงค่าเลขฐาน 2 ที่ยาวมากๆ ให้สั้นลงและอ่านง่ายขึ้น โดย 1 หลักของเลขฐาน 16 แทนได้ 4 บิตของเลขฐาน 2 (เช่น F16=11112) มักใช้ในการแสดงที่อยู่หน่วยความจำหรือรหัสคำสั่งเครื่อง

4. **คำถาม:** จงแสดงวิธีการแปลงเลขฐานต่อไปนี้:
   - แปลง 29_{10} เป็นเลขฐาน 2 และฐาน 16
   - แปลง 11011_2 เป็นเลขฐาน 10 และฐาน 8

   แนวทางการเฉลย:

   29_{10} เป็นฐาน 2:

   - 29÷2=14 เศษ 1
   - 14÷2=7 เศษ 0
   - 7÷2=3 เศษ 1
   - 3÷2=1 เศษ 1
   - 1÷2=0 เศษ 1

   อ่านจากล่างขึ้นบน: 11101_2

   29_{10} เป็นฐาน 16:

   - 29÷16=1 เศษ 13(D)
   - 1÷16=0 เศษ 1
   - อ่านจากล่างขึ้นบน: $1D_{16}$

   11011_2 เป็นฐาน 10:

   - $1\times2^4+1\times2^3+0\times2^2+1\times2^1+1\times2^0$
   - $=16+8+0+2+1=27_{10}$

11011_2 เป็นฐาน 8:

แบ่งกลุ่ม 3 บิตจากขวา: 011 011_2 (เติม 0 ข้างหน้าถ้าไม่ครบ)

$011_2=3_8$

$011_2=3_8$

รวมเป็น: 33_8

5. **คำถาม:** จงอธิบายความหมายของ บิต (Bit) และ ไบต์ (Byte) พร้อมระบุความสัมพันธ์ระหว่างหน่วยทั้งสอง และยกตัวอย่างการใช้งานเพื่อวัดขนาดข้อมูล

   แนวทางการเฉลย:

   - **บิต (Bit):** มาจากคำว่า "Binary Digit" เป็นหน่วยข้อมูลที่เล็กที่สุดในคอมพิวเตอร์ มีค่าได้เพียง 2 สถานะ คือ 0 หรือ 1
   - **ไบต์ (Byte):** เป็นกลุ่มของบิตที่รวมกันเพื่อแทนข้อมูลที่มีความหมาย มักใช้แทนตัวอักษร 1 ตัว โดย **1 ไบต์ มีค่าเท่ากับ 8 บิต**
   - **ความสัมพันธ์และการใช้งาน:**
     - บิตใช้ระบุสถานะพื้นฐาน เช่น สถานะของหลอดไฟ (เปิด/ปิด), True/False
     - ไบต์ใช้ในการวัดขนาดของข้อมูลที่เป็นตัวอักษร, ไฟล์เอกสาร, หรือหน่วยความจำ เช่น ไฟล์เอกสารขนาด 10 KB (กิโลไบต์) หมายถึงมีขนาดประมาณ 10,000 ไบต์ หรือจอภาพสี 24 บิต หมายถึงแต่ละพิกเซลใช้ 24 บิตในการเก็บข้อมูลสี (8 บิตสำหรับแดง, 8 บิตสำหรับเขียว, 8 บิตสำหรับน้ำเงิน)

6. **คำถาม:** จงอธิบายวิธีการลบเลขฐานสอง 1010_2-0101_2 (หรือ $10_{10}-5_{10}$) โดยใช้วิธี **1's Complement** และ **2's Complement** พร้อมแสดงขั้นตอนอย่างละเอียดในแต่ละวิธี

   แนวทางการเฉลย: วิธี 1's Complement:

   - ตัวตั้ง: 1010_2
   - ตัวลบ: 0101_2

     1. หา 1's Complement ของตัวลบ (0101_2): ได้ 1010_2

2. นำตัวตั้งมาบวกกับ 1's Complement ของตัวลบ:

   1010_2

   $+1010_2$

10100_2 (มีตัวทด 1)

3. นำตัวทด (End-Around Carry) ไปบวกเพิ่มที่บิตขวาสุดของผลลัพธ์:

   0100_2

   $+1_2$

0101_2

- **ผลลัพธ์:** 0101_2 (เท่ากับ 5_{10})

วิธี 2's Complement:

- ตัวตั้ง: 1010_2
- ตัวลบ: 0101_2

1. หา 2's Complement ของตัวลบ (0101_2):

   - 1's Complement ของ 0101_2 คือ 1010_2
   - บวก 1: $1010_2+1_2=1011_2$

2. นำตัวตั้งมาบวกกับ 2's Complement ของตัวลบ:

   1010_2 $+1011_2$

10101_2 (มีตัวทด 1)

3. ทิ้งตัวทดที่บิตซ้ายสุด (Discard Final Carry)

- **ผลลัพธ์:** 0101_2 (เท่ากับ 5_{10})

7. **คำถาม:** จงอธิบายหลักการใช้งานของรหัส ASCII ในการแทนข้อมูลประเภทตัวเลข ตัวอักษร และสัญลักษณ์ โดยกล่าวถึงความแตกต่างระหว่าง Standard ASCII และ Extended ASCII

   แนวทางการเฉลย:

   - **หลักการใช้งาน ASCII:** รหัส ASCII (American Standard Code for Information Interchange) เป็นมาตรฐานการเข้ารหัสที่กำหนดค่าตัวเลขให้กับตัวอักษร (เช่น 'A', 'a'), ตัวเลข ('0' ถึง '9'), และสัญลักษณ์ต่างๆ (เช่น '!', '@', '#') เพื่อให้คอมพิวเตอร์สามารถเข้าใจและประมวลผลข้อมูลเหล่านี้ในรูปแบบเลขฐานสองได้ เมื่อผู้ใช้ป้อนตัวอักษร คอมพิวเตอร์จะแปลงเป็นรหัส ASCII ที่เป็นเลขฐานสองเพื่อจัดเก็บ/ประมวลผล และเมื่อแสดงผล ก็จะแปลงรหัสฐานสองกลับมาเป็นตัวอักษร
   - **Standard ASCII (ASCII มาตรฐาน):**
     - ใช้ **7 บิต** ในการแทนค่า ทำให้สามารถแทนตัวอักษรได้ 27=128 ตัว (ตั้งแต่รหัส 0-127)
     - ครอบคลุมตัวอักษรพื้นฐานภาษาอังกฤษ (A-Z, a-z), ตัวเลข (0-9), สัญลักษณ์ทางคณิตศาสตร์, เครื่องหมายวรรคตอน และรหัสควบคุมต่างๆ
   - **Extended ASCII (ASCII เพิ่มเติม/ขยาย):**
     - ใช้ **8 บิต** ในการแทนค่า ทำให้สามารถแทนตัวอักษรได้ 28=256 ตัว (ตั้งแต่รหัส 0-255)
     - เพิ่มเติมจาก Standard ASCII โดยใช้บิตที่ 8 เพื่อเพิ่มสัญลักษณ์พิเศษ, ตัวอักษรที่มีเครื่องหมายกำกับ (accent marks), สัญลักษณ์กราฟิก, หรือตัวอักษรของภาษาอื่นๆ อย่างไรก็ตาม Extended ASCII มีหลายเวอร์ชัน (Code Page) ซึ่งอาจทำให้เกิดปัญหาความไม่เข้ากันได้ของรหัสเมื่อใช้งานต่างระบบกัน